第5章 ARMv8处理器体系结构

5.1 计算机系统概述

5.2 ARMv8体系结构

5.3 鲲鹏920系列处理器

5.4 TaiShan 200服务器

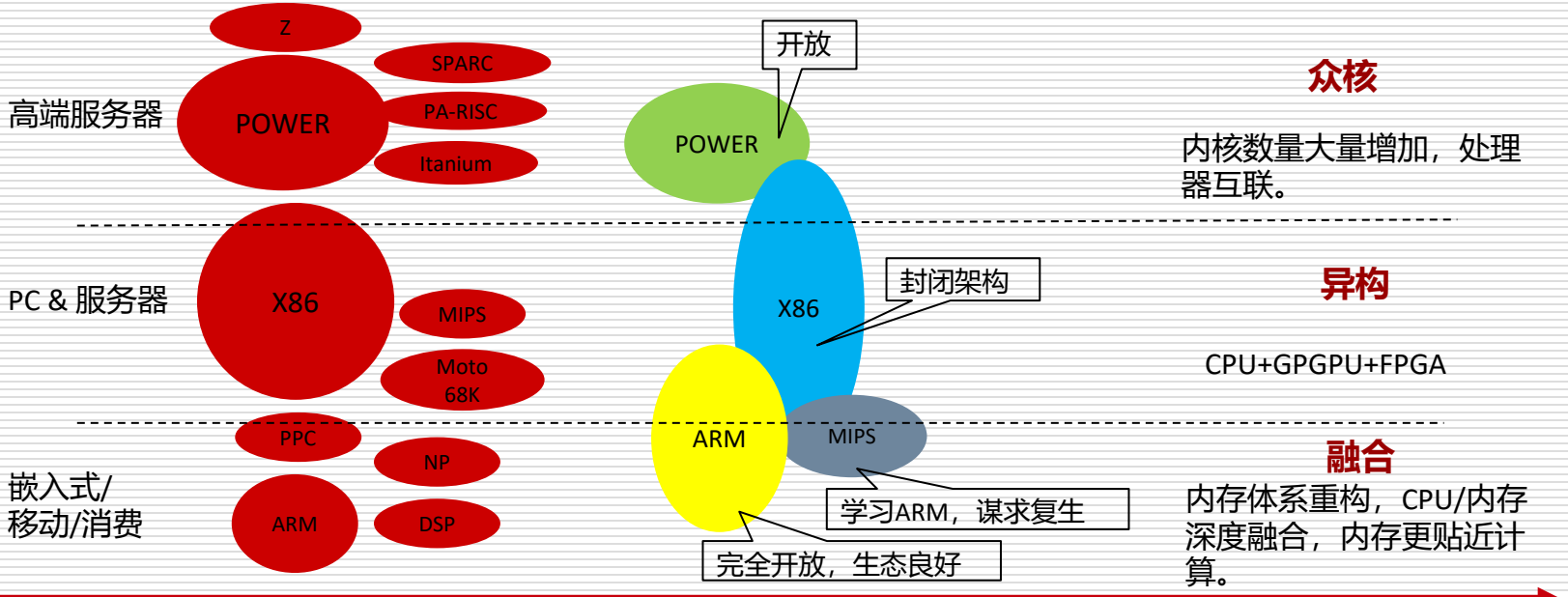
第5章 ARMv8处理器体系结构

5.1 计算机系统概述

5.1.1 处理器发展

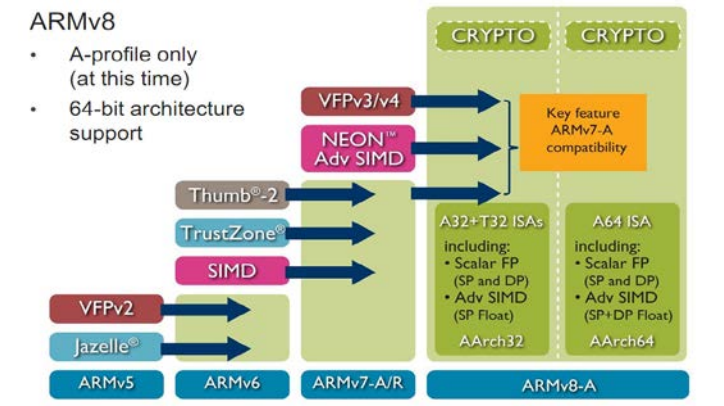
5.1.2 ARM架构发展史

处理器发展趋势


高端服务器
PC & 服务器
嵌入式/
移动/消费
Z
SPARC
POWER
PA-RISC
Itanium
X86
MIPS
Moto
68K
PPC
NP
ARM
DSP
开放
POWER
X86
封闭架构
ARM
MIPS
学习ARM，谋求复生
完全开放，生态良好
众核
内核数量大量增加，处理器互联。
异构
CPU+GPGPU+FPGA
融合
内存体系重构，CPU/内存深度融合，内存更贴近计算。
过去：架构众多，百花齐放
现在：生态成熟，架构垄断
未来：摩尔定律失效，寻求多方向突破

指令集：RISC vs CISC

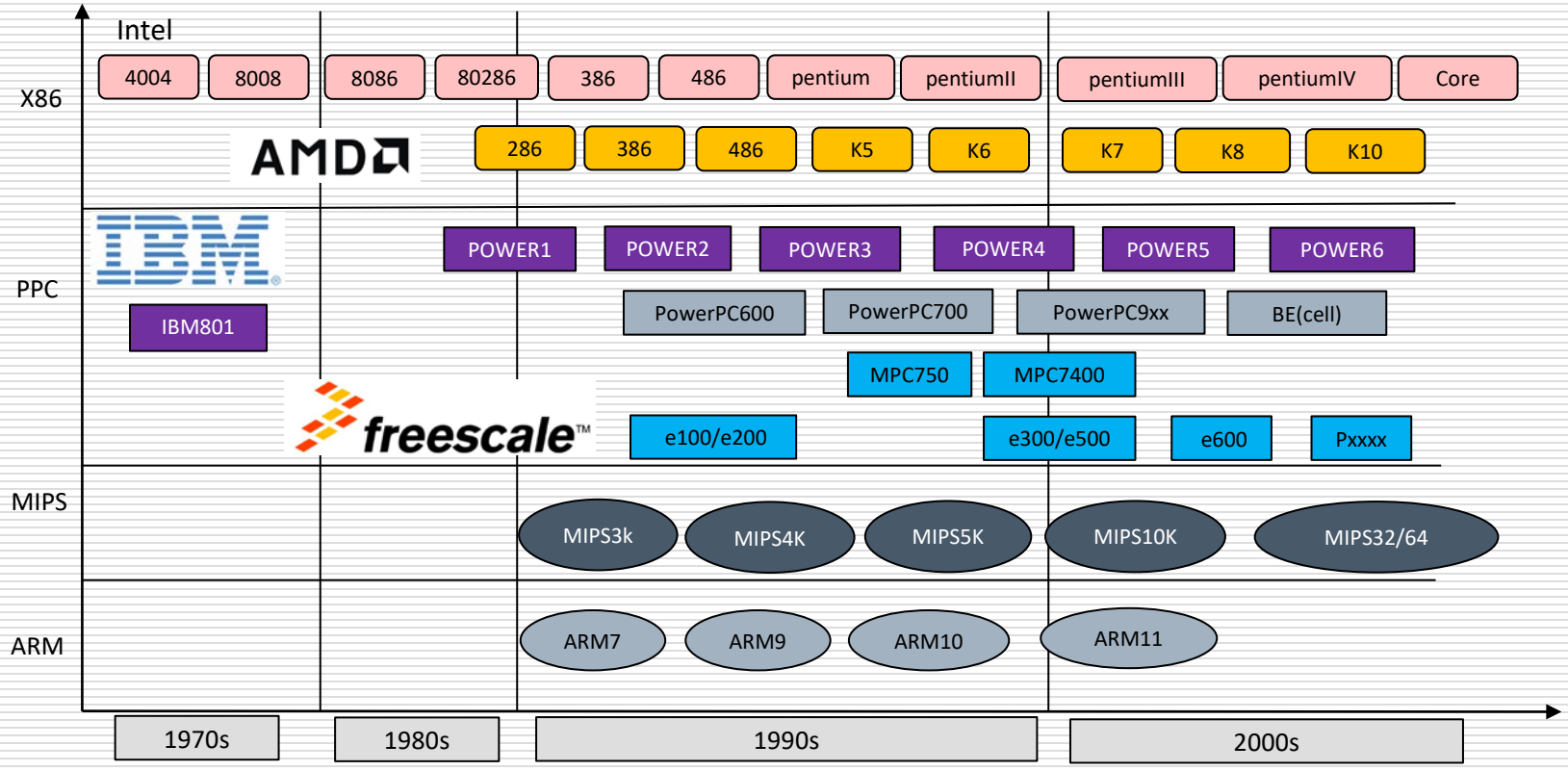
- ARM：使用精简指令集（RISC），大幅简化架构，仅保留所需要的指令，可以让整个处理器更为简化，拥有小体积、高效能的特性；ARMv8架构支持64位操作，指令32位，寄存器64位，寻址能力64位；指令集使用NEON扩展结构；
- X86：使用复杂指令集（CISC)，以增加处理器本身复杂度为代价，换取更高的性能；X86指令集从MMX，发展到了SSE，AVX；



ARMv8架构的特点：

- 31个64位通用寄存器，原来架构只有15个通用寄存器；
- 新指令集支持64位运算，指令中的寄存器编码由4位扩充到5位；
- 新指令集仍然是32位，减少了条件执行指令，条件执行指令的4位编码释放出来用于寄存器编码；
- 堆栈指针SP和程序指针PC都不再是通用寄存器了，同时推出了零值寄存器（类似PowerPC的r0）；
- A64与A32的高级SIMD和FP相同；
- 高级SIMD与VFP共享浮点寄存器，支持128位宽的vector；
- 新增加解密指令。

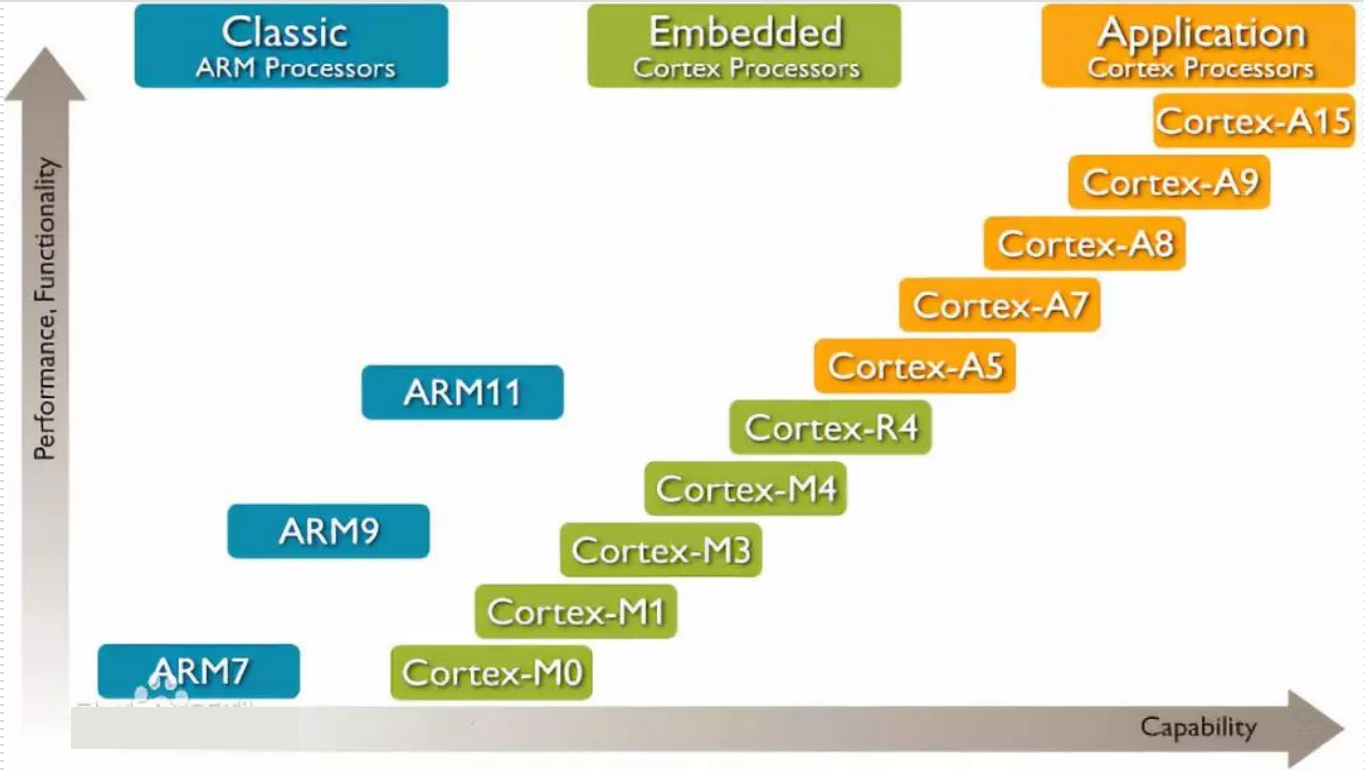
主流CPU发展路径


Intel
X86
4004
8008
8086
80286
386
486
pentium
pentiumII
pentiumIII
pentiumIV
Core
AMD
286
386
486
K5
K6
K7
K8
K10
IBM
PPC
POWER1
POWER2
POWER3
POWER4
POWER5
POWER6
IBM801
PowerPC600
PowerPC700
PowerPC9xx
BE(cell)
MPC750
MPC7400
freescale™
e100/e200
e300/e500
e600
Pxxxx
MIPS
MIPS3k
MIPS4K
MIPS5K
MIPS10K
MIPS32/64
ARM
ARM7
ARM9
ARM10
ARM11
1970s
1980s
1990s
2000s

ARM架构发展史

架构	处理器家族
ARMv1	ARM1
ARMv2	ARM2、ARM3
ARMv3	ARM6、ARM7
ARMv4	StrongARM、ARM7TDMI、ARM9TDMI
ARMv5	ARM7EJ、ARM9E、ARM10E、XScale
ARMv6	ARM11、ARM Cortex-M
ARMv7	ARM Cortex-A、ARM Cortex-M、ARM Cortex-R
ARMv8	Cortex-A50[9]

ARM架构发展史

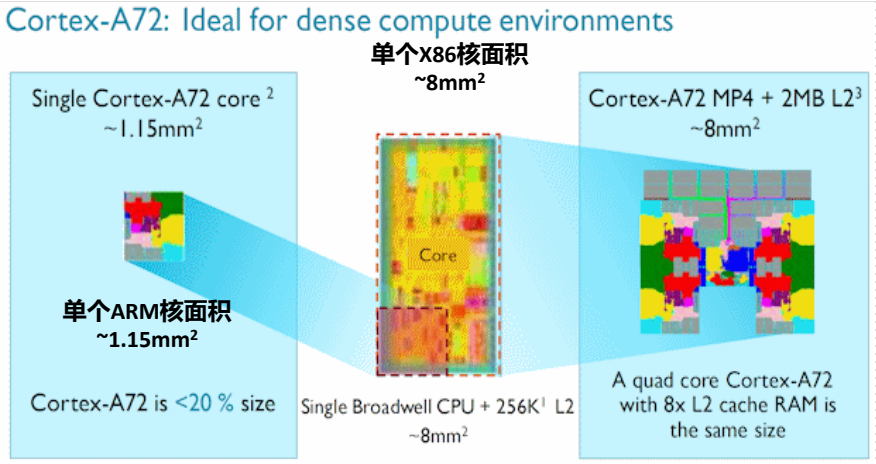

Classic
ARM Processors
Embedded
Cortex Processors
Application
Cortex Processors
Cortex-A15
Cortex-A9
Cortex-A8
Cortex-A7
Cortex-A5
ARM11
Cortex-R4
Cortex-M4
ARM9
Cortex-M3
Cortex-M1
ARM7
Cortex-M0
Performance, Functionality
Capability

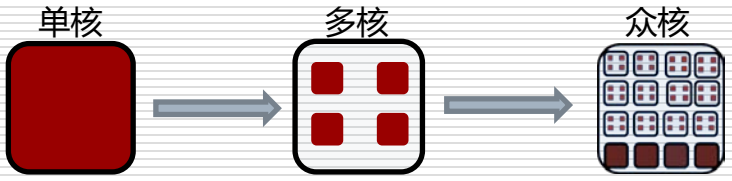
ARM架构发展史

- ARM处理器的分类：
  - ARM经典处理器（Classic Processors）；
  - ARM Cortex应用处理器；
    - 面向复杂操作系统和用户应用的Cortex-A（Applications，应用）系列
    - 针对实时处理和控制应用的Cortex-R（Real-time，实时）系列
    - 针对微控制器与低功耗应用优化的Cortex-M（Microcontroller）系列
  - ARM Cortex嵌入式处理器；
  - ARM专业处理器

ARM提供更多计算核心

- 工艺、主频遇到瓶颈后，开始通过增加核数的方式来提升性能；
- 芯片的物理尺寸有限制，不能无限制的增加；
- ARM的众核横向扩展空间优势明显。





ARM公司授权体系

- ARM目前在全球拥有大约1000个授权合作商、320家伙伴，但是购买架构授权的厂家不超过20家，中国有华为、飞腾获得了架构授权。



第5章 ARMv8处理器体系结构

5.2 ARMv8体系结构

5.2.1 ARMv7体系结构

5.2.2 ARMv8体系结构

第5章 ARMv8处理器体系结构

5.2 ARMv8体系结构

5.2.1 ARMv7体系结构

ARM微处理器的工作状态

ARM体系结构的存储器格式

ARM体系结构的指令长度及数据宽度

ARM微处理器的处理器模式

ARM体系结构的寄存器组织

ARM微处理器的异常状态

ARM体系结构

字、半字、字节

字（Word）

在ARM体系结构中，字的长度为32位

半字（Half-Word）

在ARM体系结构中，半字的长度为16位

字节（Byte）

在ARM体系结构中，字节的长度为8位。

字对齐：四字节对齐

半字对齐：两字节对齐

ARM体系结构

ARM微处理器的工作状态

两种工作状态：

- ARM状态：处理器执行32位的字对齐的ARM指令
- Thumb状态：处理器执行16位的、半字对齐的Thumb指令

ARMv7体系结构

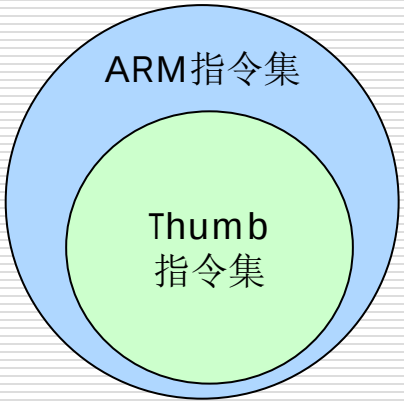
- 在程序执行的过程中，处理器可以在两种状态下切换
- ARM指令集和Thumb指令集都有相应的状态切换命令BX。
- ARM处理器在开始执行代码时，只能处于ARM状态，在异常中断中也只能处于ARM状态。

ARMv7体系结构

状态切换的一个例子

使用BX指令将ARM内核的操作状态在ARM状态和Thumb状态之间进行切换。

从ARM状态切换到Thumb状态的程序代码如下：



指令集关系

```
        CODE32
        LDR     R0, =Lable+1
        BX      R0
        CODE16
Lable   MOV     R1, #12
```

程序代码

跳转地址标号

地址最低位为0，表示切换到ARM状态

执行完BX指令，处理器切换到ARM状态，开始执行ARM指令

ARMv7体系结构

ARM微处理器的处理器模式

7种处理器模式:

- 用户模式（usr）：　　用户应用程序
- 系统模式（sys）：　　特权模式
- 快速中断模式（fiq）：用于快速数据传输
- 中断模式（irq）：　　通用的中断处理
- 管理模式（svc）：　　操作系统使用的保护模式
- 终止模式(abt):　　数据访问中止或指令预取中止
- 未定义指令模式（und）：未定义的指令执行时

特权模式 = 异常模式 + 系统模式

管理模式（svc）是系统复位后的默认模式

特权模式（Privileged Modes）

异常模式（Exception Modes）

ARMv7体系结构

ARM微处理器的处理器模式

ARM微处理器的运行模式可以通过软件改变，也可以通过外部中断或异常处理改变。

大多数的应用程序运行在用户模式下，当处理器运行在用户模式下时，某些被保护的系统资源是不能被访问的。也不能改变模式。除非异常发生。

特权模式可以自由地访问系统资源和改变模式(通过软件改变)。

ARMv7体系结构

ARM体系结构的寄存器组织

37个32位的寄存器:

- 31个通用寄存器

  未分组寄存器（Unbanked Register）：R0～R7

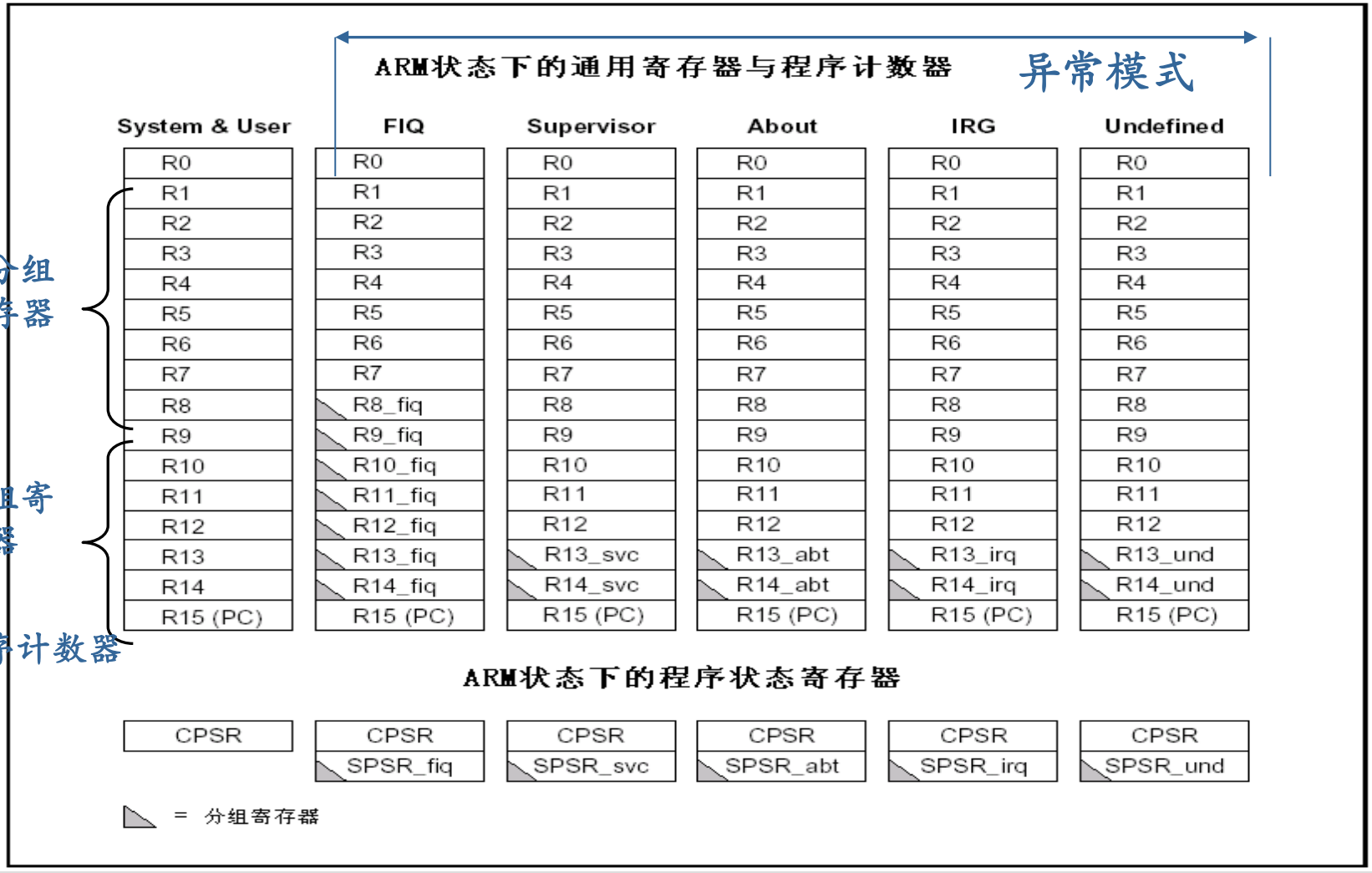
  分组寄存器(Banked Register）：R8～R14

  程序计数器R15（PC指针）

- 6个状态寄存器，用以标识CPU的工作状态及程序的运行状态
- 在每一种处理器模式下均有一组相应的寄存器与之对应。

  在所有的寄存器中，有些是在7种处理器模式下共用的同一个物理寄存器，而有些寄存器则是在不同的处理器模式下有不同的物理寄存器

ARMv7体系结构


ARM状态下的通用寄存器与程序计数器
异常模式
System & User
FIQ
Supervisor
About
IRG
Undefined
R0
R1
R2
R3
R4
R5
R6
R7
R8
R8_fiq
R9
R9_fiq
R10
R10_fiq
R11
R11_fiq
R12
R12_fiq
R13
R13_svc
R13_abt
R13_irq
R13_und
R14
R14_fiq
R14_svc
R14_abt
R14_irq
R14_und
R15 (PC)
未分组寄存器
分组寄存器
程序计数器PC
ARM状态下的程序状态寄存器
CPSR
SPSR_fiq
SPSR_svc
SPSR_abt
SPSR_irq
SPSR_und
= 分组寄存器

ARMv7体系结构

ARM状态下的寄存器组织

- 未分组寄存器（Unbanked Register）R0～R7

  同一个寄存器名，在ARM微处理器内部只有一个独立的物理寄存器与之对应。

  在所有的运行模式下，未分组寄存器都指向同一个物理寄存器，他们未被系统用作特殊的用途。

  (在中断或异常处理进行运行模式转换时，由于不同的处理器运行模式均使用相同的物理寄存器，可能会造成寄存器中数据的破坏，这一点在进行程序设计时应引起注意。)

ARMv7体系结构

- 分组寄存器(Banked Register) R8～R14

ARM状态下的通用寄存器与程序计数器

System & User	FIQ	Supervisor	About	IRG	Undefined
R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7
R8	R8_fiq	R8	R8	R8	R8
R9	R9_fiq	R9	R9	R9	R9
R10	R10_fiq	R10	R10	R10	R10
R11	R11_fiq	R11	R11	R11	R11
R12	R12_fiq	R12	R12	R12	R12
R13	R13_fiq	R13_svc	R13_abt	R13_irq	R13_und
R14	R14_fiq	R14_svc	R14_abt	R14_irq	R14_und
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)

ARM状态下的程序状态寄存器

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	SPSR_fiq	SPSR_svc	SPSR_abt	SPSR_irq	SPSR_und

= 分组寄存器

ARMv7体系结构

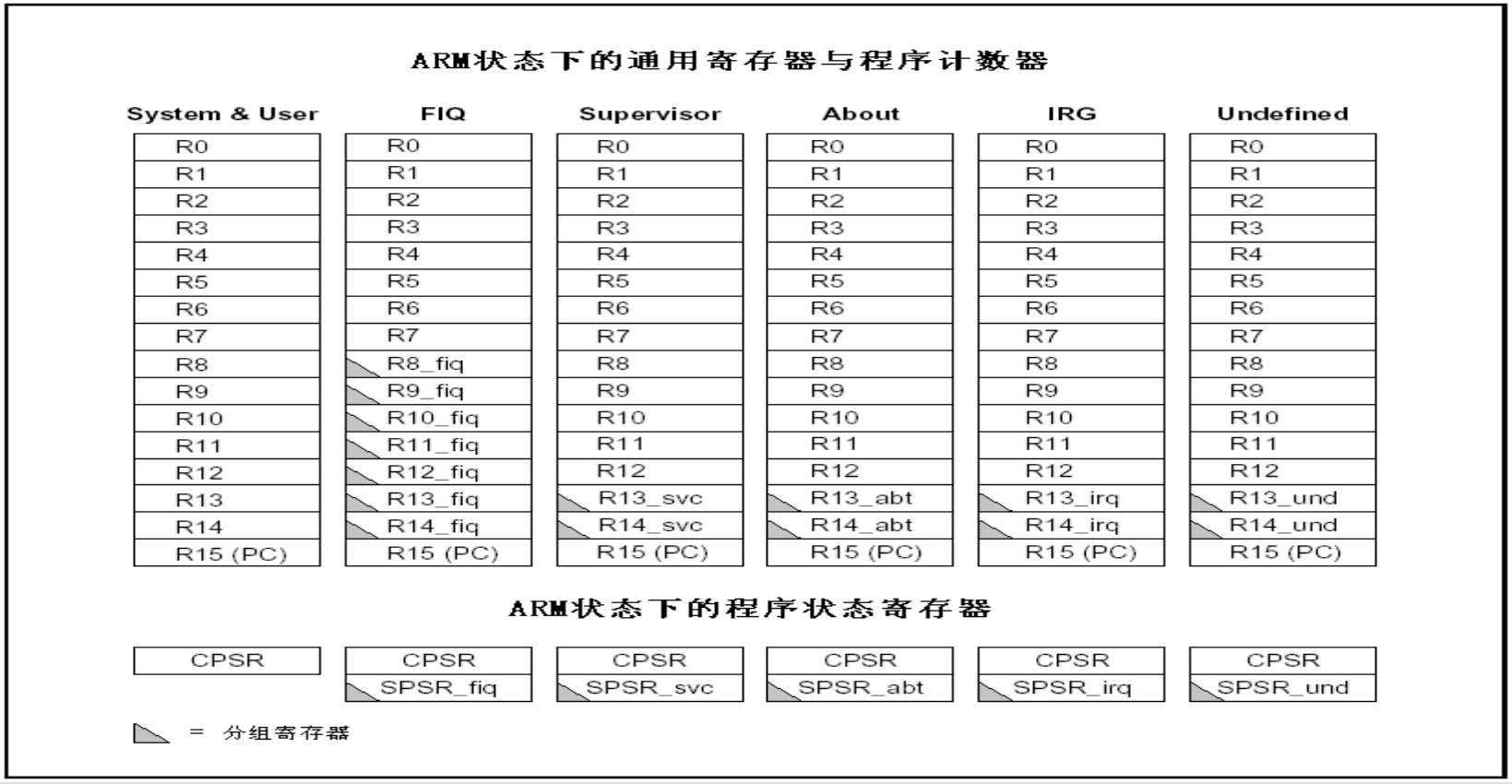
fiq模式:R8_fiq～R12_fiq;非fiq模式: R8_usr～R12_usr



ARMv7体系结构

R13、R14：对应6个不同的物理寄存器(SP堆栈，LR连接)

用户模式与系统模式共用一个



ARMv7体系结构

- 寄存器R13

  寄存器R13在ARM指令中常用作堆栈指针。

  在用户应用程序的初始化部分，一般都要初始化每种模式下的R13 （R13_svc、R13_irq、R13_fiq、R13_abt和R13_und ），使其指向该运行模式的栈空间。

  当程序的运行进入异常模式时，可以将需要保护的寄存器放入R13所指向的堆栈，而当程序从异常模式返回时，则从对应的堆栈中恢复，采用这种方式可以保证异常发生后程序的正常执行。

ARMv7体系结构

- 寄存器R14

  子程序连接寄存器（Subroutine Link Register）或链接寄存器LR。

  子程序调用：

  当执行BL（BLX）子程序调用指令时，R14中得到R15（程序计数器PC）的备份。

  中断或异常：

  对应的分组寄存器R14_svc、R14_irq、R14_fiq、R14_abt和R14_und用来保存R15的返回值。

ARMv7体系结构

- 寄存器R14

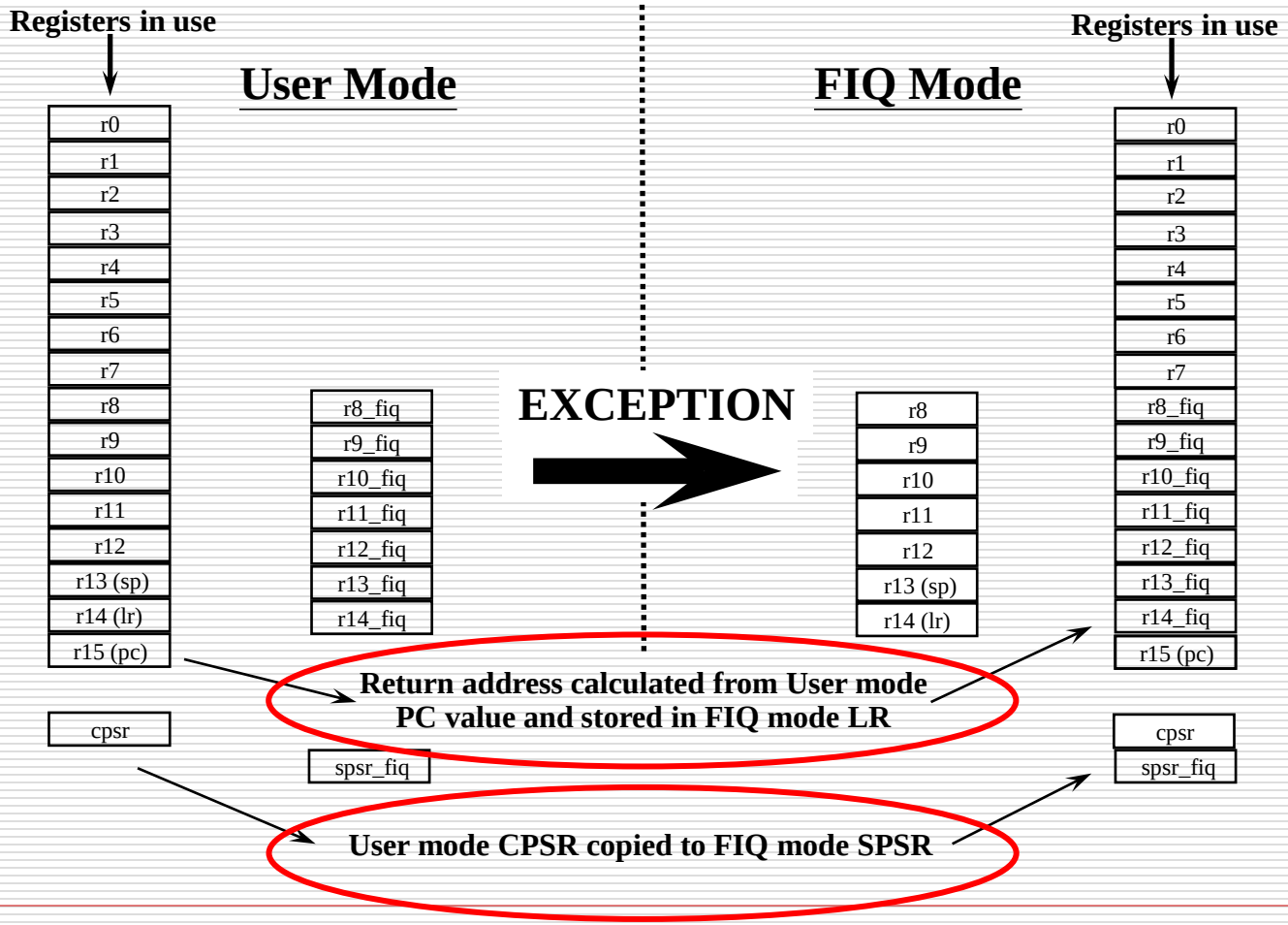
  子程序连接寄存器（Subroutine Link Register）或链接寄存器LR。

  PC——LR

  中断或异常：

  对应的分组寄存器R14_svc、R14_irq、R14_fiq、R14_abt和R14_und用来保存R15的返回值。

User to FIQ mode



ARM体系结构

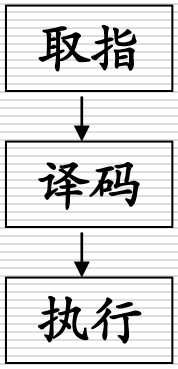
- 寄存器R15

  程序计数器PC。

  寄存器R15用作程序计数器（PC）。在ARM状态下，位[1:0]为0，位[31:2]用于保存PC；在Thumb状态下，位[0]为0，位[31:1]用于保存PC。

  由于ARM体系结构采用了多级流水线技术，对于ARM指令集而言，PC总是指向当前指令的下两条指令的地址，即PC的值为当前指令的地址值加8个字节。

ARM体系结构



从存储器取指

指令译码

从寄存器组中读寄存器，移位和ALU操作，将寄存器写回到寄存器组

指令流水线功能段划分

ARM单周期指令3级流水线操作



ARM体系结构

- 当前程序状态寄存器 CPSR

CPSR(Current Program Status Register)

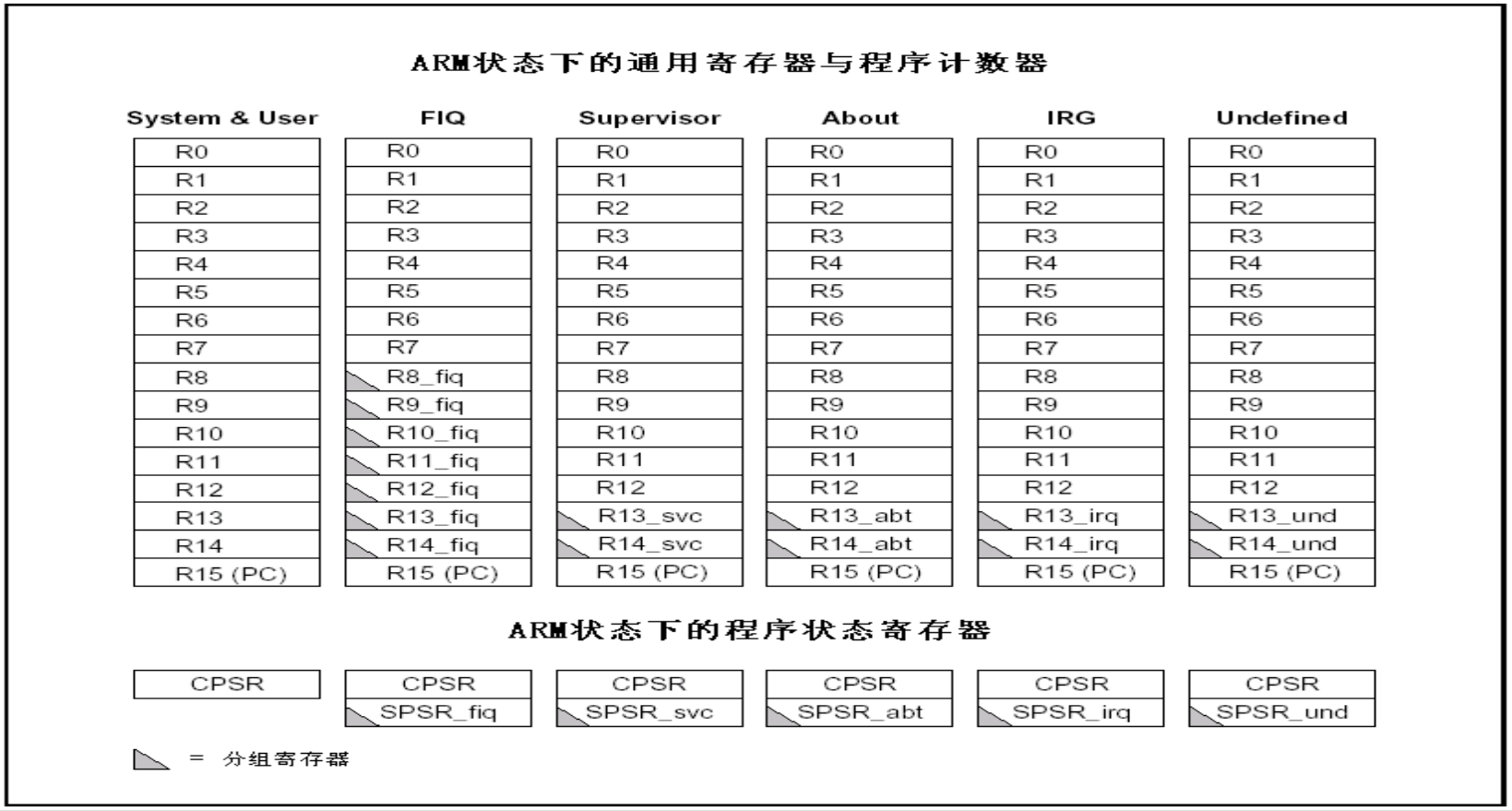
CPSR可在任何运行模式下被访问，它包括条件标志位、中断禁止位、当前处理器模式标志位，以及其他一些相关的控制和状态位。

备份的程序状态寄存器SPSR（Saved Program Status Register），当异常发生时，SPSR用于保存CPSR的当前值，从异常退出时则可由SPSR来恢复CPSR。

用户模式和系统模式不属于异常模式，他们没有SPSR，当在这两种模式下访问SPSR，结果是未知的。

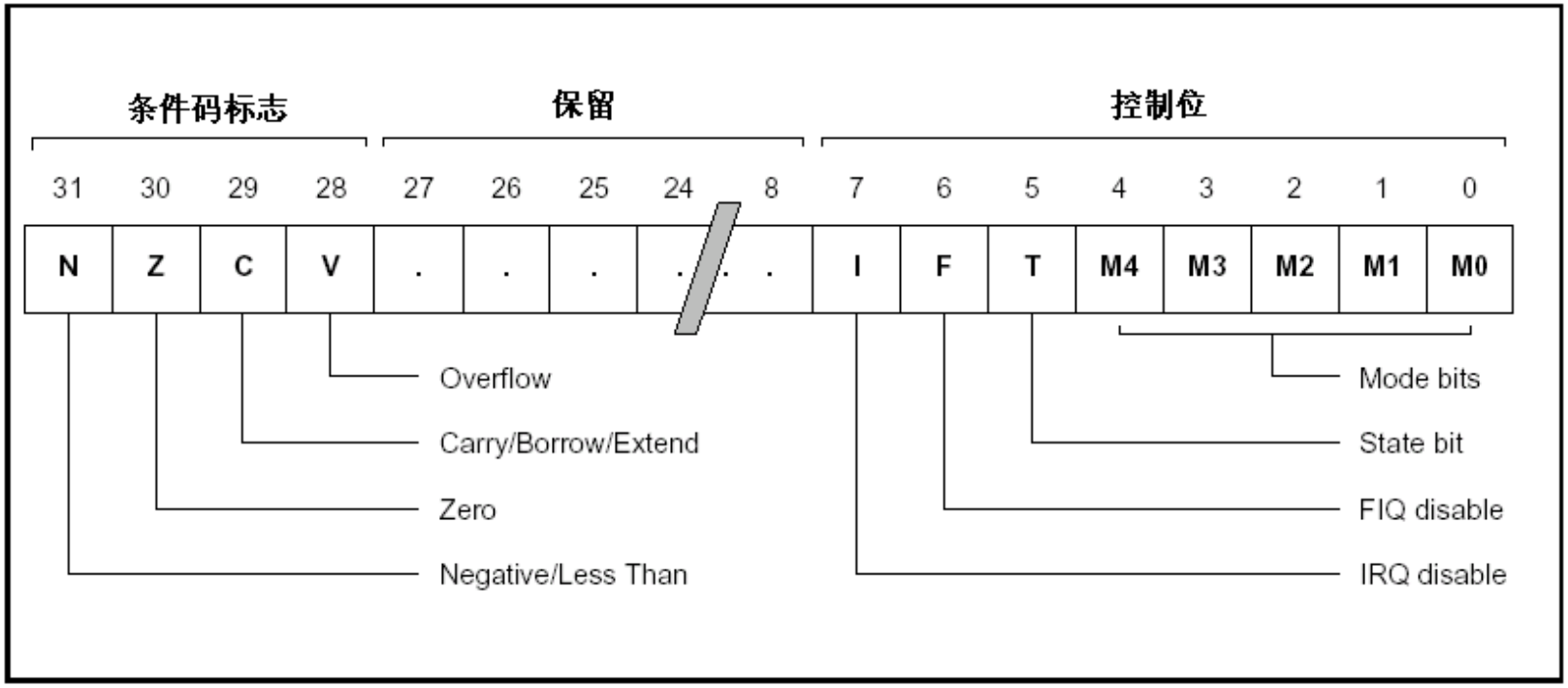
ARM体系结构

用户模式和系统模式不属于异常模式，他们没有SPSR，当在这两种模式下访问SPSR，结果是未知的。



ARM体系结构

ARM状态下的寄存器组织



CPSR

ARM体系结构

- 条件码标志（Condition Code Flags）

N、Z、C、V:条件码标志位。可被算术或逻辑运算的结果所改变，并且可以决定某条指令是否被执行。

在ARM状态下，绝大多数的指令都是有条件执行的。

在Thumb状态下，仅有分支指令是有条件执行的。

ARM体系结构

标志位	含　　义
N	N=1 表示运算的结果为负数；N=0 表示运算的结果为正数或零。
Z	Z=1 表示运算的结果为零；Z=0表示运算的结果为非零。
C	加法运算（包括比较指令CMN）：当运算结果产生了进位时（无符号数溢出），C=1，否则C=0。 减法运算（包括比较指令CMP）：当运算时产生了借位（无符号数溢出），C=0，否则C=1。 对于包含移位操作的非加/减运算指令，C为移出值的最后一位。 对于其他的非加/减运算指令，C的值通常不改变。
V	对于加/减法运算指令，当操作数和运算结果为二进制的补码表示的带符号数时，V=1表示符号位溢出。 对于其他的非加/减运算指令，C的值通常不改变。
Q	在ARM v5及以上版本的E系列处理器中，用Q标志位指示增强的DSP运算指令是否发生了溢出。在其他版本的处理器中，Q标志位无定义。

ARMv7体系结构

- 控制位

CPSR的低8位（包括I、F、T和M[4：0]）称为控制位，当发生异常时这些位可以被改变。如果处理器处于特权模式，这些位也可以由程序修改。

中断禁止位I、F：

I=1　　禁止IRQ中断

F=1　　禁止FIQ中断

T标志位：

T=1　　Thumb状态

T＝0　　ARM状态

ARMv7体系结构

M[4: 0]运行模式位：这些位决定了处理器的运行模式。

M[4: 0]	处理器模式	可访问的寄存器
0b10000	用户模式	PC, CPSR, R0-R14
0b10001	FIQ模式	PC, CPSR, SPSR_fiq, R14_fiq-R8_fiq, R7～R0
0b10010	IRQ模式	PC, CPSR, SPSR_irq, R14_irq,R13_irq,R12～R0
0b10011	管理模式	PC, CPSR, SPSR_svc, R14_svc,R13_svc,R12～R0,
0b10111	中止模式	PC, CPSR, SPSR_abt, R14_abt,R13_abt,R12～R0,
0b11011	未定义模式	PC, CPSR, SPSR_und, R14_und,R13_und,R12～R0,
0b11111	系统模式	PC, CPSR, R14～R0

ARMv7体系结构

- 异常（Exception）

  当正常的程序执行流程发生改变时，称之为异常。

- 异常类型 运行模式

复位	svc
未定义指令	und
软件中断	svc
指令预取中止	abt
数据中止	abt
IRQ（外部中断请求）	irq
FIQ（快速中断请求）	fiq

ARMv7体系结构

异常类型	具体含义
复位	当处理器的复位电平有效时，产生复位异常，程序跳转到复位异常处理程序处执行。
未定义指令	当ARM处理器或协处理器遇到不能处理的指令时，产生未定义指令异常。可使用该异常机制进行软件仿真。
软件中断	该异常由执行SWI指令产生，可用于用户模式下的程序调用特权操作指令。可使用该异常机制实现系统功能调用。
指令预取中止	若处理器预取指令的地址不存在，或该地址不允许当前指令访问，存储器会向处理器发出中止信号，但当预取的指令被执行时，才会产生指令预取中止异常。
数据中止	若处理器数据访问指令的地址不存在，或该地址不允许当前指令访问时，产生数据中止异常。
IRQ（外部中断请求）	当处理器的外部中断请求引脚有效，且CPSR中的I位为0时，产生IRQ异常。系统的外设可通过该异常请求中断服务。
FIQ（快速中断请求）	当处理器的快速中断请求引脚有效，且CPSR中的F位为0时，产生FIQ异常。

ARMv7体系结构

异常是由内部或外部源产生以引起处理器处理事件。例如，外部中断或试图执行未定义指令都会引起异常。在处理异常之前，处理器状态必须保留，以便在异常处理完成之后，原来的程序能够重新执行。

在ARM体系结构中，支持7种类型的异常。分别为**软件中断**、**未定义指令陷阱**及**系统复位功能**和**外部事件等**。

异常出现后，强制从异常类型对应的固定存储器地址开始执行程序。这些固定的地址称为异常向量。

ARM体系结构

ARM处理器对异常中断的响应过程：

- 将CPSR的内容保存到将要执行的异常中断对应的SPSR中
- 设置当前状态寄存器CPSR中的相应位
- 将引起异常指令的下一条指令的地址保存到新的异常工作模式的R14
- 给程序计数器（PC）强制赋值

ARM体系结构

- 每个异常模式对应有两个寄存器R13_<mode>、R14_<mode>分别保存相应模式下的堆栈指针、返回地址；**堆栈指针可用来定义一个存储区域保存其它用户寄存器，这样异常处理程序就可以使用这些寄存器**。
- FIQ模式还有额外的专用寄存器R8_fiq～R12_fiq，使用这些寄存器可以加快快速中断的处理速度。

从异常中断处理程序中返回

- 从异常中断处理程序中返回时，需要执行以下四个基本操作
  - 所有修改过的用户寄存器必须从处理程序的保护堆栈中恢复（即出栈）。
  - 将SPSR_mode寄存器内容复制到CPSR中，使得CPSR从相应的SPSR中恢复，即恢复被中断的程序工作状态；
  - 根据异常类型将PC变回到用户指令流中相应指令处
  - 最后清除CPSR中的中断禁止标志位I/F。

异常中断向量表

- 中断向量表中**指定了各异常中断与其处理程序的对应关系**
- 每个异常中断对应的中断向量表的4个字节的空间中存放一个跳转指令或者一个向PC寄存器中赋值的数据访问指令
- 存储器的前8个字中除了地址0x00000014之外，全部被用作异常矢量地址

异常中断向量表

表 2-3 异常处理模式

异常类型	模式	正常地址	高端向量地址
复位	管理	0x00000000	0xffff0000
未定义指令	未定义	0x00000004	0xffff0004
软件中断(SWI)	管理	0x00000008	0xffff0008
预取中止(取指令存储器中止)	中止	0x0000000c	0xffff000c
数据中止(数据访问存储器中止)	中止	0x00000010	0xffff0010
IRQ(中断)	IRQ	0x00000018	0xffff0018
FIQ(快速中断)	FIQ	0x0000001c	0xffff001c

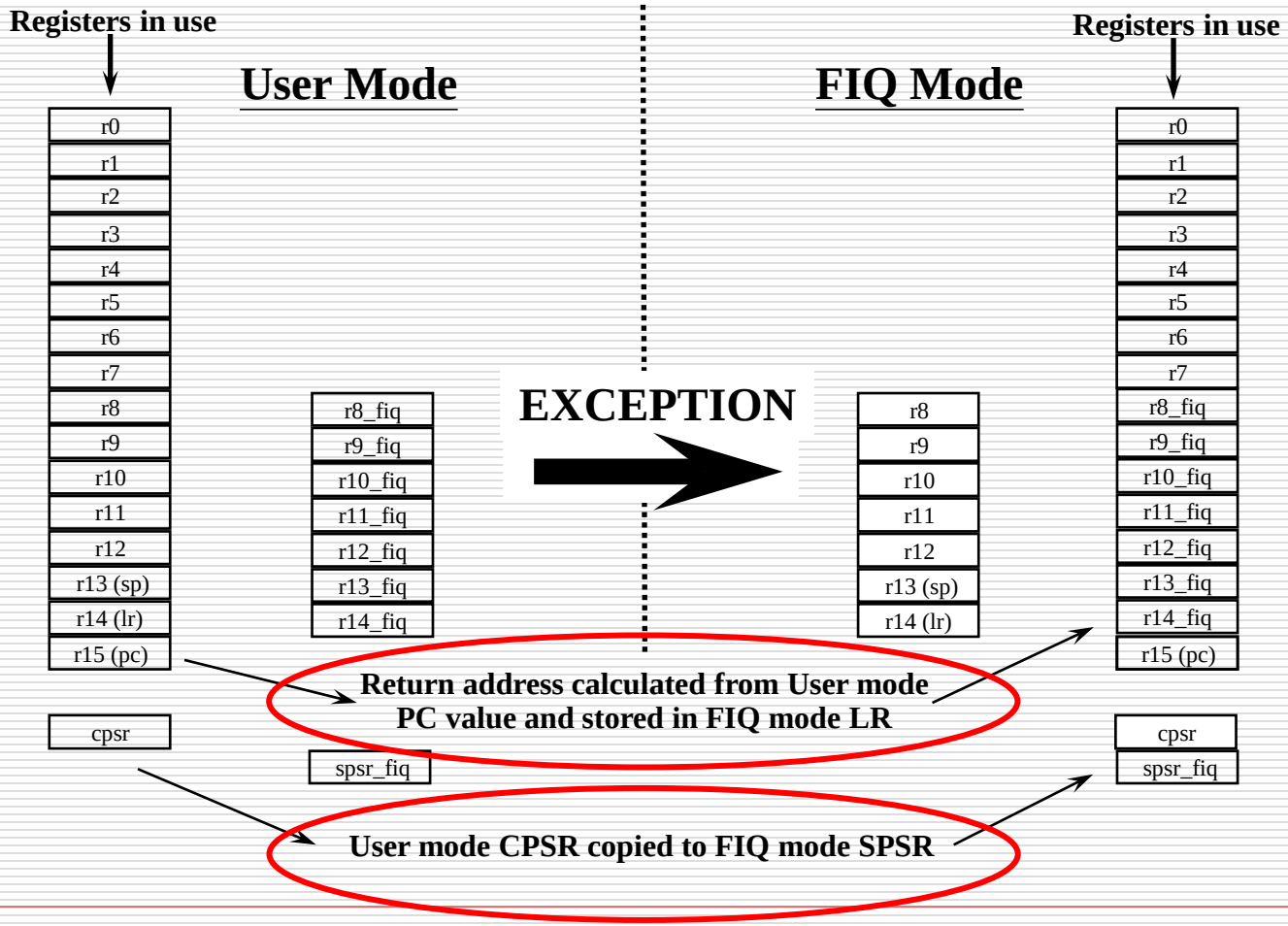
ARM体系结构

异常中断的优先级

当几个异常中断同时发生时，在ARM中通过给各异常中断赋予一定的优先级来实现处理次序

- 复位（最高优先级）；
- 数据异常中止；
- FIQ；
- IRQ；
- 预取指异常中止；
- SWI、未定义指令（包括缺协处理器）。

Example: User to FIQ mode



ARM体系结构

- 复位

复位完成下列操作：

```
R14_svc = UNPREDICTABLE value
SPSR_svc = UNPREDICTABLE value
CPSR[4:0] = 0b10011              /*进入管理模式*/
CPSR[5] = 0                      /*在ARM状态执行*/
CPSR[6] = 1                      /*禁止快速中断*/
CPSR[7] = 1                      /*禁止正常中断*/
If high vectors configured then
    PC=0xFFFF0000
else
    PC=0x00000000
```

复位后，ARM处理器在禁止中断的管理模式下，从地址0x00000000或0xffff0000开始执行指令。复位不需要返回。

ARM体系结构

- 软件中断（SWI)→管理异常

  管理异常异常出现时，执行下列操作：

```
R14_svc = address of next instruction after the SWI instruction
SPSR_svc = CPSR
CPSR[4:0] = 0b10011                /*进入管理模式*/
CPSR[5] = 0                        /*在ARM状态执行*/
                                   /*CPSR[6] 不变*/
CPSR[7] = 1                        /*禁止正常中断*/
If high vectors configured then
    PC=0xFFFF0008
else
    PC=0x00000008
```

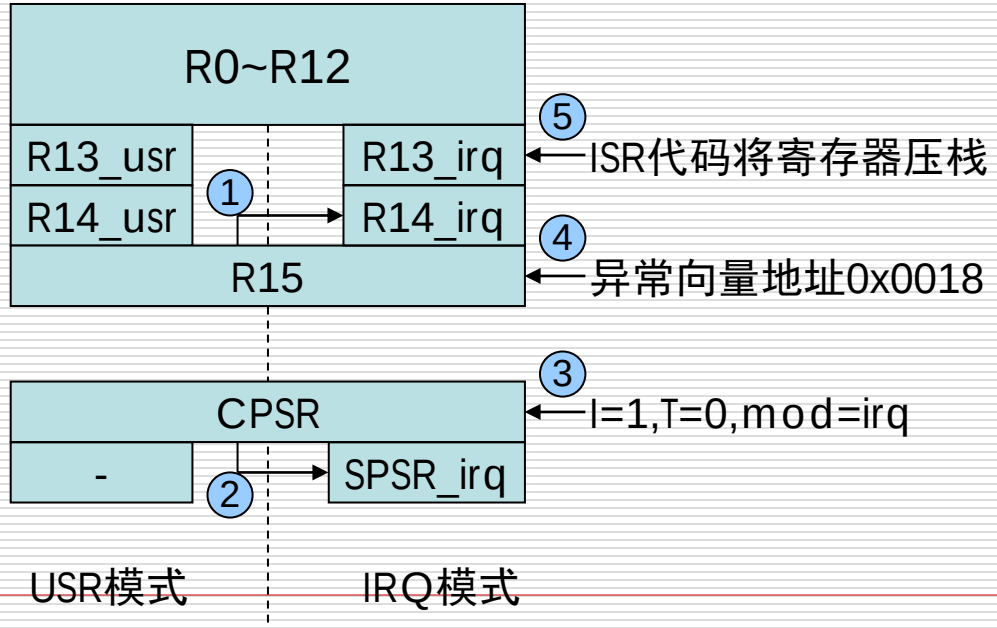
  返回：

```
MOVS PC, LR
```

ARMv7体系结构

- 进入IRQ异常模式

5 将[illegible]，跳转到IRQ模式异常服务程序入口处执行中断服务处理；



ARMv7体系结构

- 中断请求（IRQ)异常

  通过处理器上的IRQ输入引脚，由外部产生IRQ异常。IRQ异常的优先级比FIQ异常低。当进入FIQ处理时，会屏蔽掉IRQ异常。

  若CPSR的I位为1，则禁止IRQ异常。若I位为0，则ARM在指令执行完之后检查IRQ输入。

  IRQ异常出现时，执行下列操作：

```
R14_irq = address of next instruction to be executed + 4
SPSR_irq = CPSR
CPSR[4:0] = 0b10010                    /*进入IRQ模式*/
CPSR[5] = 0                  /*在ARM状态执行*/
                             /*CPSR[6] 不变*/
CPSR[7] = 1                            /*禁止正常中断*/
If high vectors configured then
    PC=0xFFFF0018
else
    PC=0x00000018
```

  返回：

```
SUBS PC, LR,#4
```

ARMv7体系结构

- 快速中断请求（FIQ)异常

  通过处理器上的FIQ输入引脚，由外部产生FIQ异常。

  若CPSR的F位为1，则禁止FIQ异常。若F位为0，则ARM在指令执行完之后检查FIQ输入。

  FIQ异常出现时，执行下列操作:

```
R14_fiq = address of next instruction to be executed + 4
SPSR_fiq = CPSR
CPSR[4:0] = 0b10001              /*进入FIQ模式*/
CPSR[5] = 0                      /*在ARM状态执行*/
CPSR[6] = 1                      /*禁止快速中断*/
CPSR[7] = 1                      /*禁止正常中断*/
If high vectors configured then
   PC=0xFFFF001C
else
   PC=0x0000001C
```

  返回:

```
SUBS PC, LR,#4
```

ARM体系结构

- 从异常返回

异常处理完毕之后，ARM微处理器会执行以下几步操作从异常返回：

1、将连接寄存器LR的值减去相应的偏移量后送到PC中

2、将SPSR复制回CPSR中

3、若在进入异常处理时设置了中断禁止位，要在此清除

ARM体系结构

- 异常进入/退出小结

	返回指令	
BL	MOV PC, LR	;R14
SWI	MOVS PC, LR	;R14_svc
UDEF	MOVS PC, LR	;R14_und
FIQ	SUBS PC, LR, #4	;R14_fiq
IRQ	SUBS PC, LR, #4	;R14_irq
PABT	SUBS PC, LR, #4	;R14_abt
DABT	SUBS PC, LR, #8	;R14_abt
RESET	NA	

第5章 ARMv8处理器体系结构

5.2 ARMv8处理器体系结构

5.2.1 ARM体系结构

5.2.2 ARMv8体系结构

ARMv8 体系结构

ARMv8特点

- ARMv7-A compatibility
- VFP v3/v4
- NEON (adv SIMD) & VFP now standard32x128-bit registers (2x)Double-precision FP (now fully IEEE 754-compliant)
- AES and SHA-1/SHA-256（加密）

ARMv8 体系结构

ARMv8的架构继承以往ARMv7与之前处理器技术的基础，兼容现有的A32(ARM 32bit)指令集。

基于64bit的AArch64架构，新增A64(ARM 64bit)指令集外，也扩充了现有的A32(ARM 32bit)和T32(Thumb2 32bit）指令集，另外还新增加了CRYPTO(加密)模块支持。

ARMv8体系结构

ARMv8 提供AArch32 state和 AArch64 state

Execution State	Note
AArch32	提供 13 个 32bit 通用寄存器 R0-R12，一个 32bit PC 指针（R15）、堆栈指针 SP（R13）、链接寄存器 LR（R14）
	提供一个 32bit 异常链接寄存器 ELR，用于 Hyp mode 下的异常返回
	提供 32 个 64bit SIMD 向量和标量 floating-point 支持
	提供两个指令集 A32（32bit）、T32（16/32bit）
	兼容 ARMv7 的异常模型
AArch64	提供 31 个 64bit 通用寄存器 X0-X30（W0-W30），其中 X30 是程序链接寄存器 LR
	提供一个 64bit PC 指针、堆栈指针 SPx 、异常链接寄存器 ELRx
	提供 32 个 128bit SIMD 向量和标量 floating-point 支持
	定义 ARMv8 异常等级 ELx（x<4），x 越大等级越高，权限越大
	定义一组 PE state 寄存器 PSTATE（NZCV/DAIF/CurrentEL/SPSel 等），用于保存 PE 当前的状态信息

决定Execution State的条件

只有通过复位或者发生异常这两种方式才可以改变 Execution State
SPSR_EL1.M[4] 决定 EL0 的执行状态，为 0 =>64bit ，否则=>32bit
HCR_EL2.RW 决定 EL1 的执行状态，为 1 =>64bit ，否则=>32bit
SCR_EL3.RW 确定 EL2 or EL1 的执行状态，为 1 =>64bit ，否则=>32bit

ARMv8体系结构

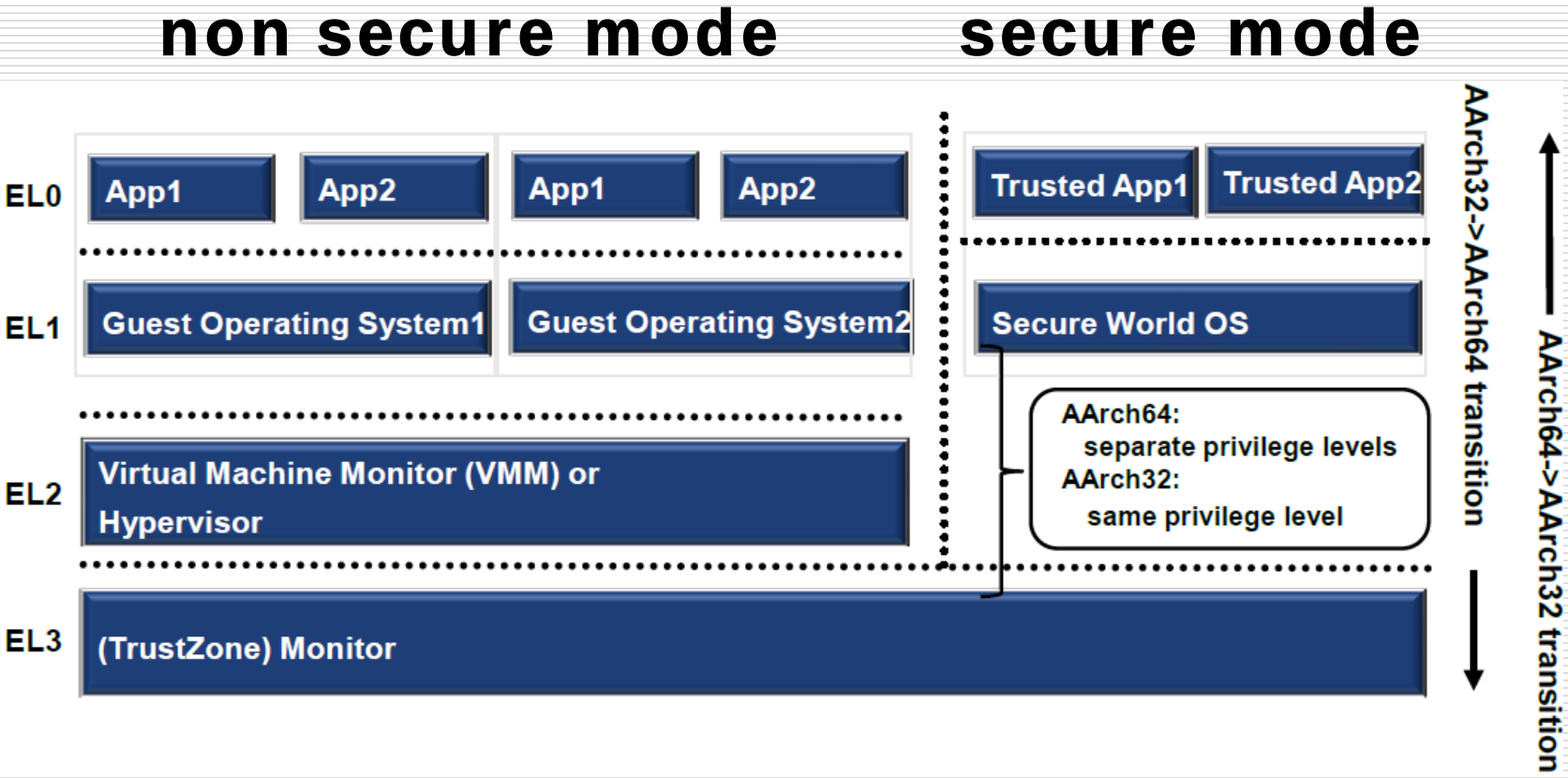
ARMv8定义EL0-EL3共 4个异常等级

EL0	Application
EL1	Linux kernel- OS
EL2	Hypervisor （可以理解为上面跑多个虚拟 OS）
EL3	Secure Monitor （ARM Trusted Firmware）

4个异常等级与安全性

ELx（x<4），x 越大等级越高，执行特权越高
执行在 EL0 称为非特权执行
EL2 没有 Secure state，只有 Non-secure state
EL3 只有 Secure state，实现 EL0/EL1 的 Secure 和 Non-secure 之间的切换
EL0 & EL1 必须要实现，EL2/EL3 则是可选实现

ARMv8体系异常等级



ARMv8体系结构

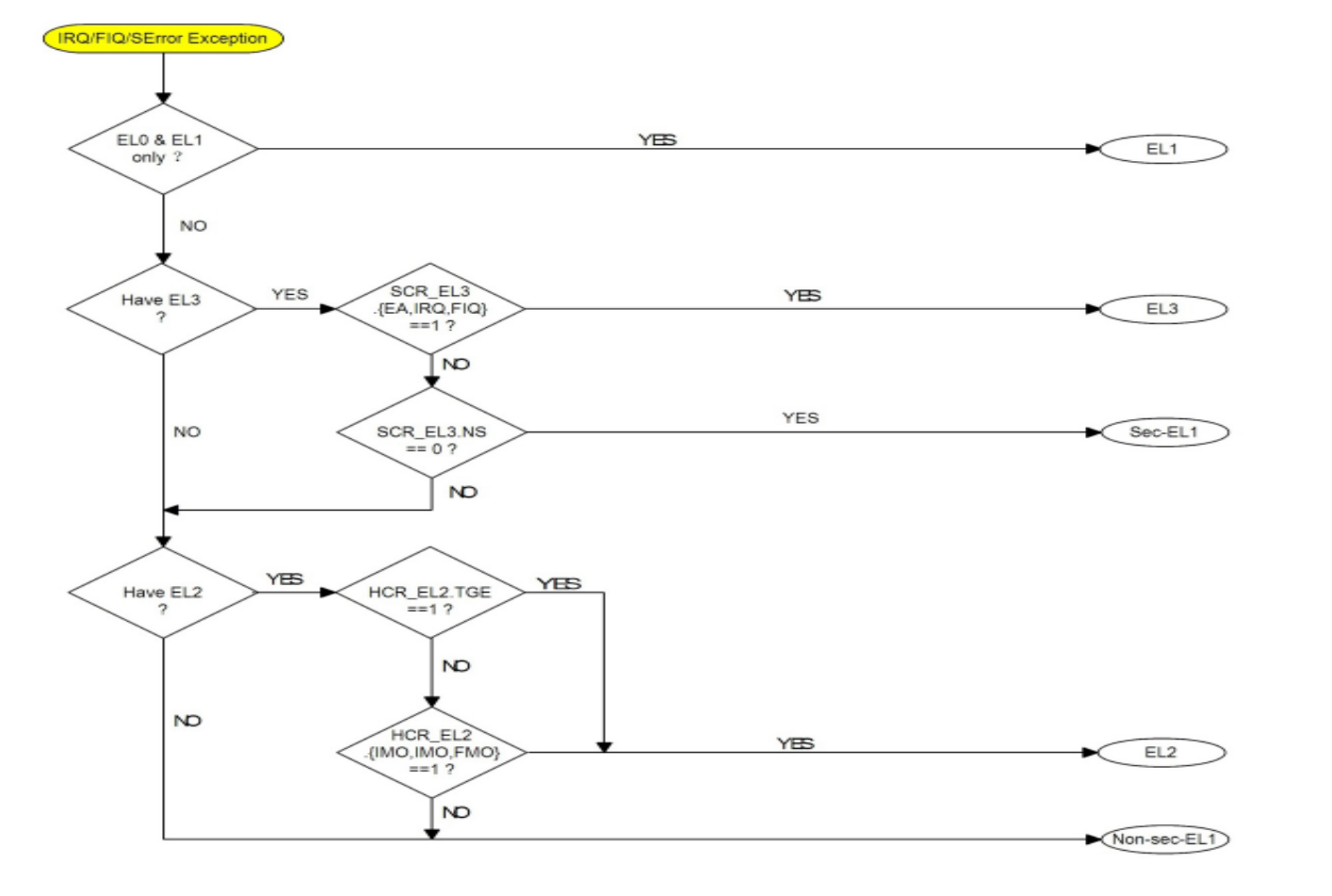
路由控制:当中断或错误时，在工作不同异常等级的路由

规则小结如下：

若 SPSR_EL1.M[4] == 0，则决定 EL0 使用 AArch64,否则 AArch32
若 SCR_EL3.RW == 1，则决定 EL2/EL1 是使用 AArch64，否则 AArch32
若 SCR_EL3.{EA, FIQ, IRQ} == 1，则所有相应的 SError\FIQ\IRQ 中断都被路由到 EL3
若 HCR_EL2.RW == 1，则决定 EL1 使用 AArch64，否则使用 AArch32
若 HCR_EL2.{AMO, IMO, FMO} == 1，则 EL1/EL0 所有对应的 SError\FIQ\IRQ 中断都被路由到 EL2,同时使能对应的虚拟中断 VSE,VI,VF
若 HCR_EL2.TGE == 1，那么会忽略 HCR_EL2.{AMO, IMO, FMO}的具体值，直接当成 1 处理，则 EL1/EL0 所有对应的 SError\FIQ\IRQ 中断都被路由到 EL2，同时禁止所有虚拟中断
注意： SCR_EL3.{EA, FIQ, IRQ}bit 的优先级高于 HCR_EL2.{AMO, IMO, FMO} bit 优先级，路由优先考虑 SCR_EL3

ARMv8体系结构

IRQ/FIQ/SError路由流程图



ARMv8体系结构

AArch64重要寄存器

寄存器类型	Bit	描述
X0-X30	64bit	通用寄存器，如果有需要可以当做 32bit 使用：W0-W30
LR （X30）	64bit	通常称 X30 为程序链接寄存器，保存跳转返回信息地址
SP_ELx	64bit	若 PSTATE.M[0] ==1，则每个 ELx 选择 SP_ELx，否则选择同一个 SP_EL0
ELR_ELx	64bit	异常链接寄存器，保存异常进入 ELx 的异常地址（x={0,1,2,3}）
PC	64bit	程序计数器，俗称 PC 指针，总是指向即将要执行的下一条指令
SPSR_ELx	32bit	寄存器，保存进入 ELx 的 PSTATE 状态信息
NZCV	32bit	允许访问的符号标志位
DIAF	32bit	中断使能位：D-Debug，I-IRQ，A-SError，F-FIQ ，逻辑 0 允许
CurrentEL	32bit	记录当前处于哪个 Exception level
SPSel	32bit	记录当前使用 SP_EL0 还是 SP_ELx，x= {1,2,3}
HCR_EL2	32bit	HCR_EL2.{TEG,AMO,IMO,FMO,RW}控制 EL0/EL1 的异常路由 逻辑 1 允许
SCR_EL3	32bit	SCR_EL3.{EA,IRQ,FIQ,RW}控制 EL0/EL1/EL2 的异常路由 逻辑 1 允许
ESR_ELx	32bit	保存异常进入 ELx 时的异常综合信息，包含异常类型 EC 等.
VBAR_ELx	64bit	保存任意异常进入 ELx 的跳转向量基地址 x={0,1,2,3}
PSTATE		不是一个寄存器，是保存当前 PE 状态的一组寄存器统称，其中可访问寄存器有： PSTATE.{NZCV,DAIF,CurrentEL,SPSel}，属于 ARMv8 新增内容，64bit 下代替 CPSR

ARMv8体系结构

AArch32 ⇔ AArch64 Register Mapping



ARMv8体系结构

AArch64 Mapping

64-bit General Purpose Register file used for:

- Scalar Integer computation
  - 32-bit and 64-bit
- Address computation
  - 64-bit

Media Register File used for:

- Scalar Single and Double Precision FP
  - 32-bit and 64-bit
- Advanced SIMD for Integer and FP
  - 64- or 128-bit wide vectors
- Cryptography

X0	X8	X16	X24
X1	X9	X17	X25
X2	X10	X18	X26
X3	X11	X19	X27
X4	X12	X20	X28
X5	X13	X21	X29
X6	X14	X22	X30*
X7	X15	X23	

V0	V8	V16	V24
V1	V9	V17	V25
V2	V10	V18	V26
V3	V11	V19	V27
V4	V12	V20	V28
V5	V13	V21	V29
V6	V14	V22	V30
V7	V15	V23	V31

ARMv8体系结构

- 异常（Exceptions）是现代处理器必备的程序随机切换机制。异常是由外部事件引起的中断服务过程，异常也用于处理需要特权软件权限才能处理的系统事件。
- ARMv8-A的异常类型：
  - 同步异常（Synchronous exception）
  - 异步异常（Asynchronous exception）

    异常直接由执行指令或者尝试执行指令引起，并且异常返回地址指明了引起异常的特定指令的细节，则该异常被定义为同步异常，否则，则称为异步异常。

    异步异常是由IRQ、FIQ这两个中断请求管脚引起的中断以及系统错误引起的异常，相应地被分为三类：IRQ、FIQ和SError（System Error，系统错误）

ARMv8体系结构

AArch64异常类型

可分为同步异常 & 异步异常两大类，如下表描述：

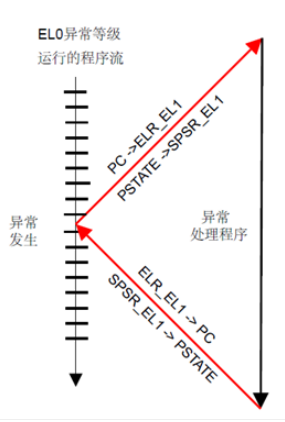
Synchronous（同步异常）	
异常类型	描述
Undefined Instruction	未定义指令异常
Illegal Execution State	非常执行状态异常
System Call	系统调用指令异常（SVC/HVC/SMC）
Misaligned PC/SP	PC/SP 未对齐异常
Instruction Abort	指令终止异常
Data Abort	数据终止异常
Debug exception	软件断点指令/断点/观察点/向量捕获/软件单步 等 Debug 异常

Asynchronous（异步异常）	
类型	描述
SError or vSError	系统错误类型，包括外部数据终止
IRQ or vIRQ	外部中断 or 虚拟外部中断
FIQ or vFIQ	快速中断 or 虚拟快速中断

ARMv8体系结构

□ ARMv8-A的异常处理：

- 首先更新备份程序状态寄存器SPSR_ELn，以保存异常处理结束返回时恢复现场必须的PSTATE信息。
- 用新的处理器状态信息更新程序状态PSTATE。如果需要，通过此步可以提升异常等级。
- 将异常处理结束返回的地址保存在异常链接寄存器ELR_ELn中。

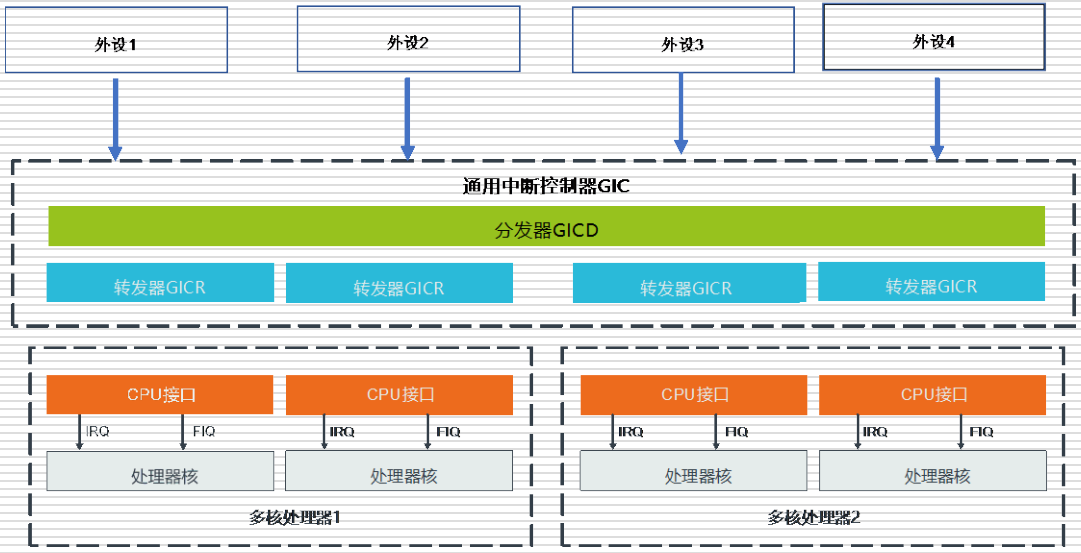


ARMv8体系结构

□AArch64的异常向量与异常向量表:

每个异常等级都有其自身的异常向量表，因而共有EL3、EL2和EL1三个异常向量表。每个异常等级都有其相应的向量基址寄存器VBAR（Vector Base Address Register），指明在该异常等级的异常向量表的基地址。

每个异常等级的异常向量表实际上有4组，每组给出的四个异常入口分别对应同步异常、IRQ、FIQ和系统错误这四种异常类别。至于应该选择哪一组的异常向量，则取决于异常是发生于当前异常等级还是更低的异常等级、异常将使用哪一个堆栈指针（SP0还是Spn）以及异常状态所处的执行状态（AArch64或AArch32）等因素。



ARMv8体系结构

ARMv8-A的异常处理:

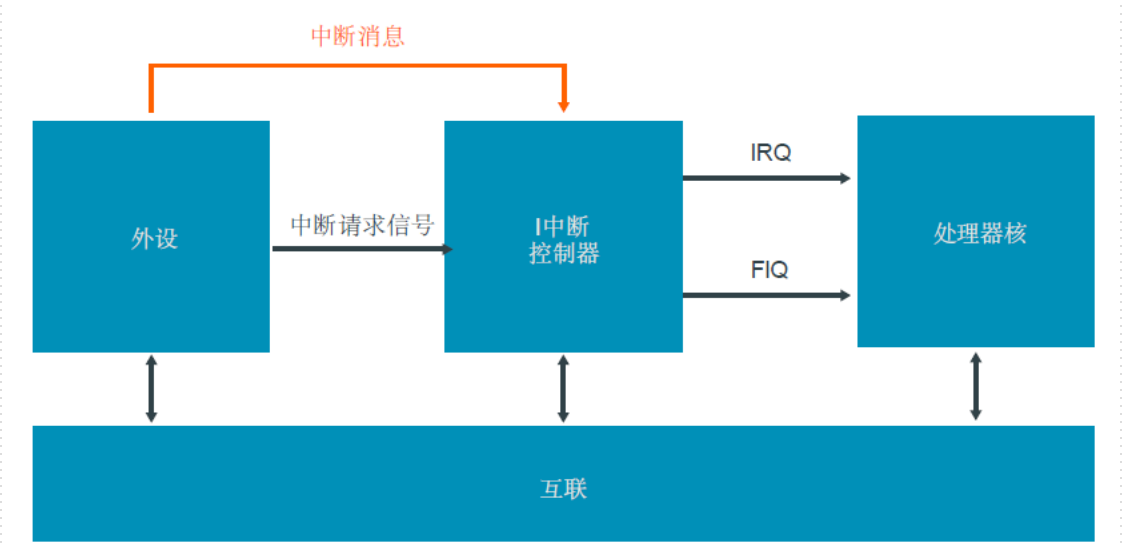
异常进入满足以下条件	向量地址偏移表			
	Synchronous(同步异常)	IRQ|| vIRQ	FIQ|| vFIQ	SError|| vSError
SP => SP_EL0 && 从Current EL来	0x000	0x080	0x100	0x180
SP => SP_Elx && 从Current EL来	0x200	0x280	0x300	0x380
64bit => 64bit && 从Low level EL来	0x400	0x480	0x500	0x580
32bit => 64bit && 从Low level EL来	0x600	0x680	0x700	0x780

- SP => SP_EL0,表示使用SP_EL0堆栈指针，由PSTATE.SP == 0决定,PSTATE.SP == 1 则SP_ELx;
- 32bit => 64bit 是指发生异常时PE从AArch32切换到AArch64的情况;

ARMv8体系结构

ARMv8-A的通用中断控制器架构：

- 对于高性能、复杂的处理器而言，外部中断源应该是相当多的，而ARM处理器核本身只能支持FIQ和IRQ两级外部中断请求输入，因而在ARM架构下，系统通过**通用中断控制器**（Generic Interrupt Controller，**GIC**）实现中断请求的仲裁、优先级排队和向处理器中断申请等操作。



ARMv8体系结构

- 中断控制器处理的中断源分为四种类型：
  - 共享外设中断（Shared Peripheral Interrupt，SPI）：外设的这类中断请求可以被连接到任何一个处理器核。
  - 私有外设中断（Private Peripheral Interrupt，PPI）：只属于某一个处理器核的外设的中断请求，例如通用定时器的中断请求。
  - 软件产生的中断（Software Generated Interrupt，SGI）：由软件写入中断控制器内的SGI寄存器引发的中断请求，通常用于处理机间通信；
  - 特定位置外设中断（Locality-specific Peripheral Interrupt，LPI）：边沿触发的基于消息的中断，其编程模式与其他类中断源完全不同。

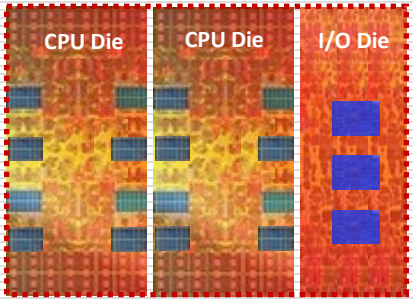
第5章 ARMv8处理器体系结构

5.3 鲲鹏处理器920

5.4 TaiShan200服务器

鲲鹏920简介

鲲鹏是SoC



制程工艺领先：业界领先7nm制程，多Die合封的Chiplet架构

自研多核内核：自研CPU内核算力提升50%，自研片间互联，支持多路互联

率先支持下一代网络和接口：支持8通道内存控制器和100GE端口

鲲鹏是计算平台

处理器->单机->集群，鲲鹏开放硬件平台



鲲鹏开放主板



鲲鹏服务器

完备的软件工具链，发挥鲲鹏最佳性能



分析扫描工具
代码迁移工具



性能优化工具
加速库

鲲鹏是生态应用

使能合作伙伴

- 应用
- 中间件
- 数据库
- 操作系统
- 服务器/PC

使能行业应用

- 大数据
- 分布式存储
- 高性能计算
- 原生应用
- 云服务

鲲鹏系列处理器发展历程

高效能计算

提供兼容ARM架构的高性能鲲鹏处理器、TaiShan服务器和解决方案，将高效能计算带入数据中心。

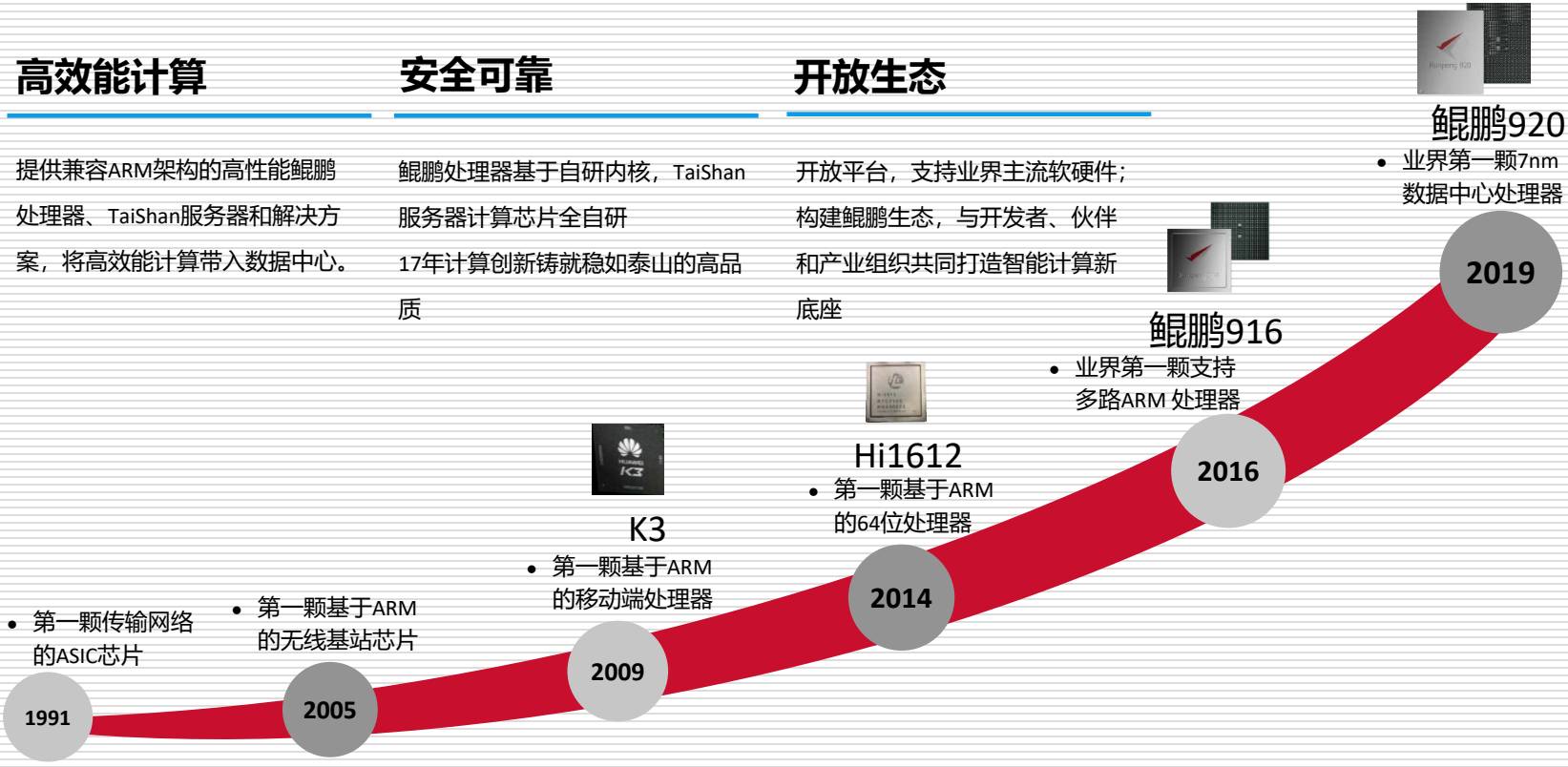
安全可靠

鲲鹏处理器基于自研内核，TaiShan服务器计算芯片全自研

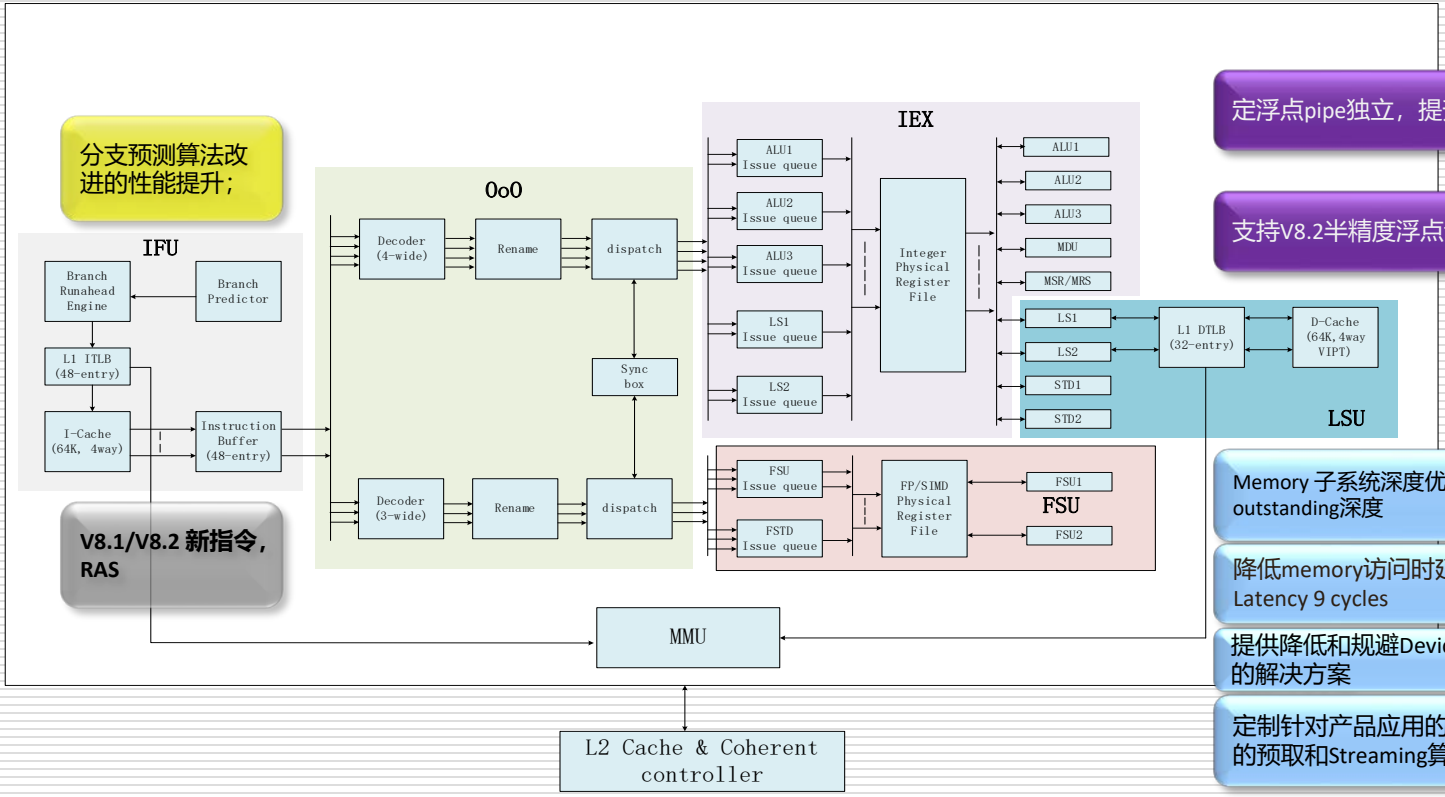
17年计算创新铸就稳如泰山的高品质

开放生态

开放平台，支持业界主流软硬件；构建鲲鹏生态，与开发者、伙伴和产业组织共同打造智能计算新底座



鲲鹏TaiShan v110处理器微架构



定浮点pipe独立，提升dispatch带宽

支持V8.2半精度浮点计算

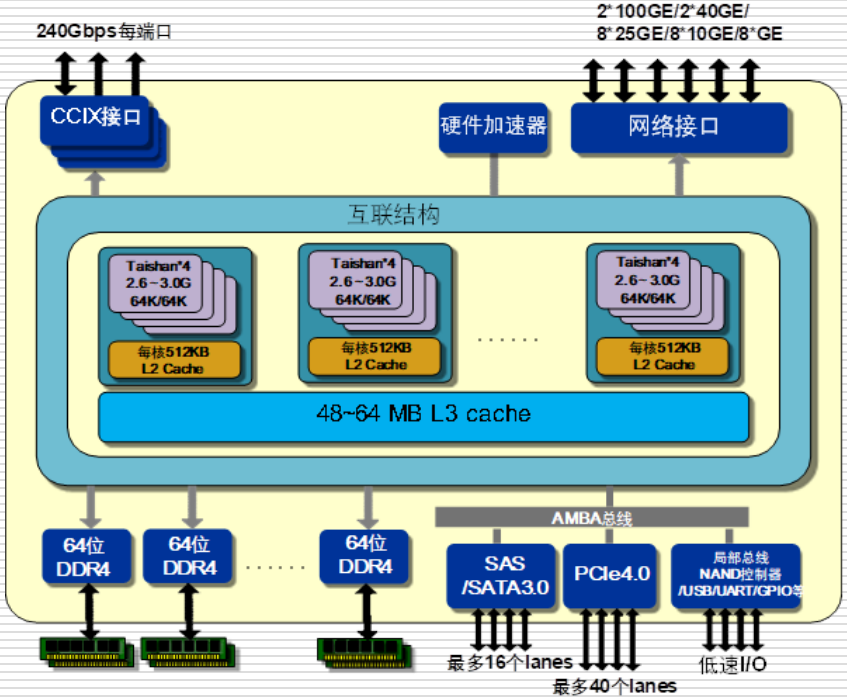
Memory 子系统深度优化，提升内存访问的outstanding深度

降低memory访问时延，每个核私有L2，Latency 9 cycles

提供降低和规避Device访问对性能影响的解决方案

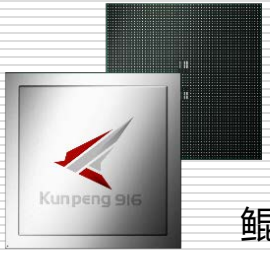
定制针对产品应用的指令和数据Cache的预取和Streaming算法

鲲鹏920系列芯片规格



- 集成最多64×自研核
  - 指令集兼容ARMv8.2, 最高主频达3.0GHz
  - 每核集成64KB L1 I/D 缓存
  - 每核独享 512KB L2 缓存，单芯片共享48-64MB L3缓存
- 8×DDR4控制器@2933MT/s
- 集成PCIe/SAS接口
  - 支持PCIe 4.0，向下兼容PCIe 3.0/2.0/1.0
  - 支持x16,x8,x4,x2,x1 PCIe 4.0, 集成20 PCIe控制器
  - 支持16×SAS/SATA 3.0控制器
- 支持CCIX接口，支持加速器的缓存一致性
- 支持2×100G RoCE v2, 支持25GE/50GE/100GE标准NIC
- 支持2P/4P扩展
- 封装大小: 60mm×75mm

鲲鹏920处理器



鲲鹏916

支持多路互联的ARM处理器

- 32核，2.4 GHz主频
- SPECint性能匹配业界中端，功耗低至75 W
- 支持4通道DDR4控制器
- 支持PCIe 3.0和SAS/SATA 3.0(磁盘)
- 集成板载GE/10 GE网络（1000M）
- 支持2路互联



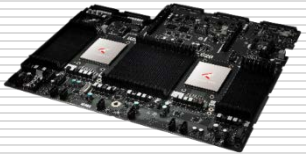
鲲鹏920

7nm制程，数据中心ARM处理器

- 计算核数提升1倍，最多64核
- SPECint性能提升超过2倍（计算机性能评估）
- 内存通道数提升1倍，支持8通道DDR4控制器
- 支持PCIe 4.0和CCIX
- 集成板载100 GE网络和加密、压缩等引擎
- 支持2路或4路互联

基于鲲鹏主板的多样化计算产品

鲲鹏主板开放共享接口与设备规范



- 主板(含鲲鹏处理器)
- 主板接口规范
- BIOS软件 & 规范
- BMC芯片/软件 & 设备管理规范

整机参考设计共享工程能



- 整机参考设计指南
- (含机箱、散热、供电等设计)
- 部件与OS兼容性列表

打造多样化计算产品

PC



服务器



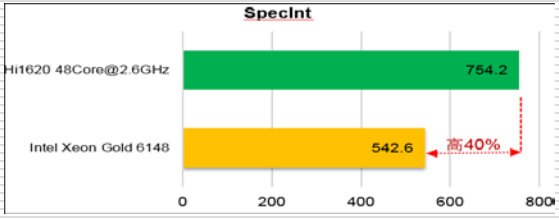
伙伴价值

- **持续供应**
- **快速开发**（3个月）
- **安全可靠**（低于业界平均故障率15%）

鲲鹏920芯片概览：高性能、高集成、异构加速

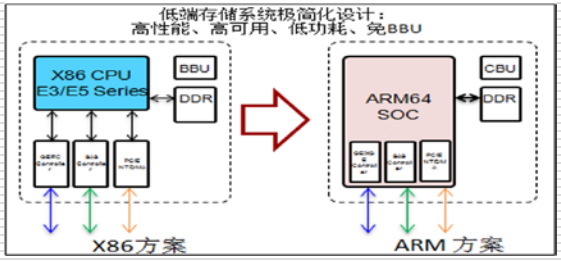
① CPU性能是关键：

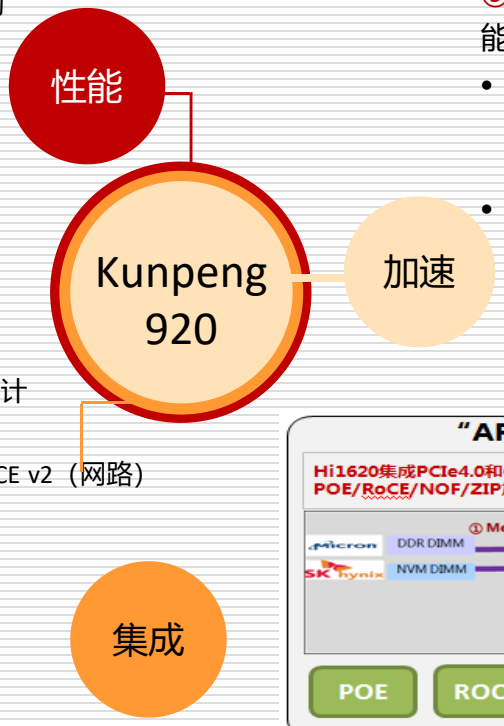
- 通过自研泰山核和多核，性能是SkyLake 6148的1.4倍，达到下一代XEON V6性能。
- 多P架构支撑产品性能扩展。
- 性能不低于友商的同时，保持26%+功耗优势。



② SoC高度集成：丰富的IO接口，支撑硬件极简设计

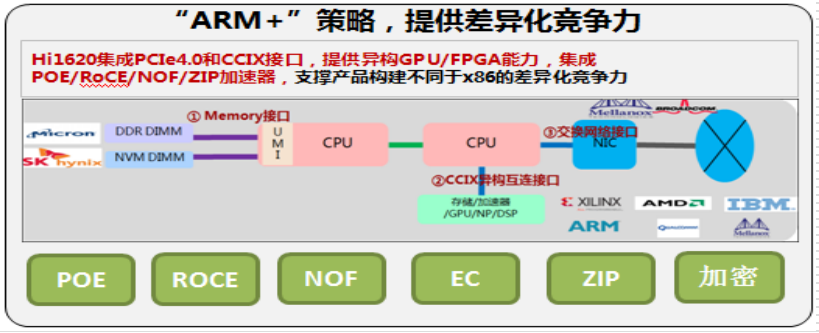
- 8 Channel DDR4。
- PCIe4.0、100GE、SAS3.0、NVME（固态）、RoCE v2（网路）





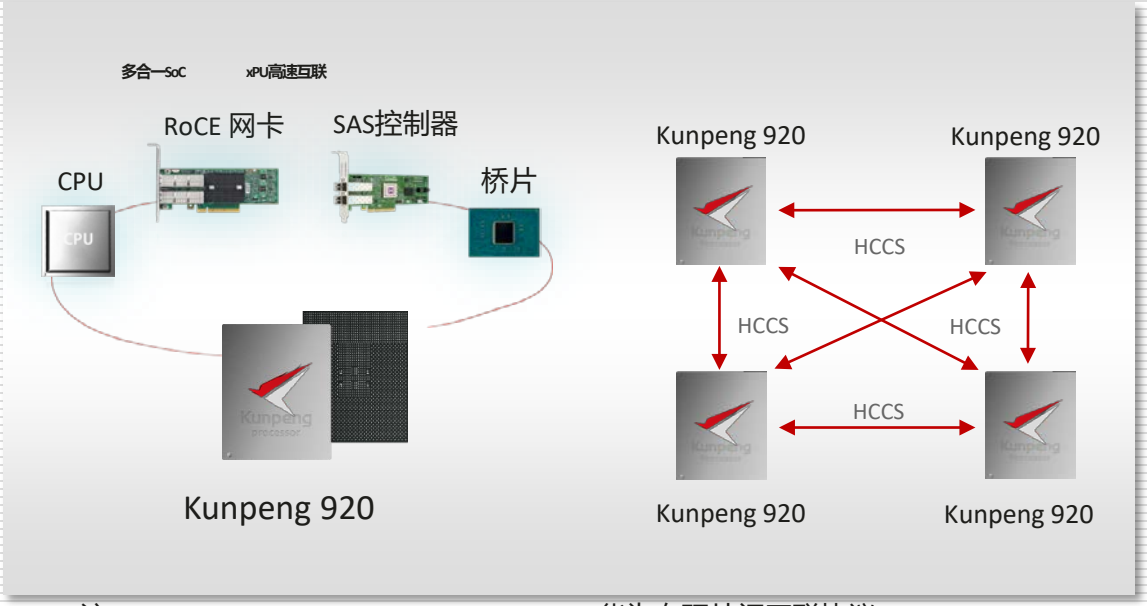
③ 芯片和产品Co-Design，架构创新：X86性能提升乏力，异构加速卸载成为常态

- 内置RAID、RSA/SEC、GZIP、EC、重删、POE加速，为产品提供In-Line业务加速，免锁队列和SSD NVMe DMA加速。
- 外接扩展加速(CCIX、SDI)， 承载存储和云业务卸载架构创新。



高集成，高能效

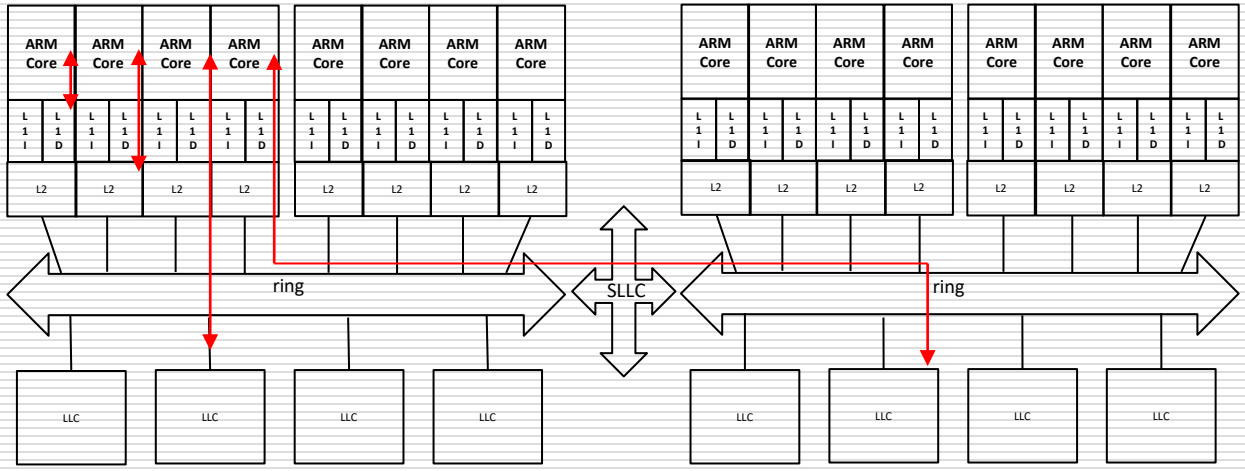
高集成



注：HCCS(Huawei Cache Coherence System) 华为自研片间互联协议

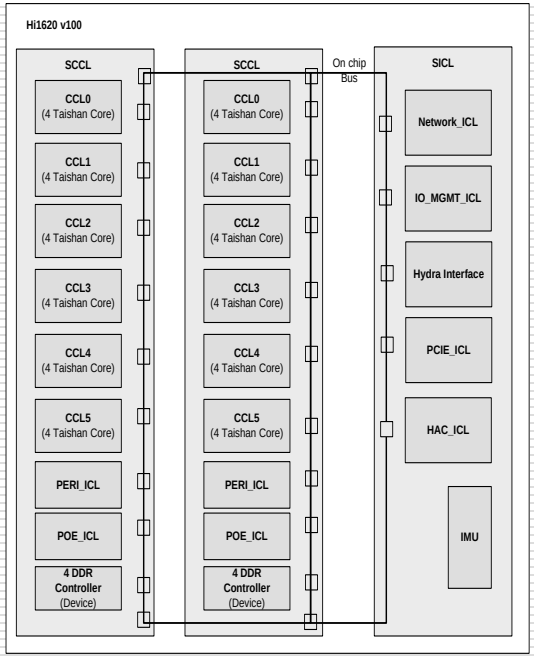
鲲鹏920系列芯片架构 – 乐高架构

- ARM 独享L1Cache和L2Cache，4个Core和1个L3 Cache tag组成一个Cluster，6~8个Cluster组成一个CPU Die，合封后的两个CPU Die共享LLC。

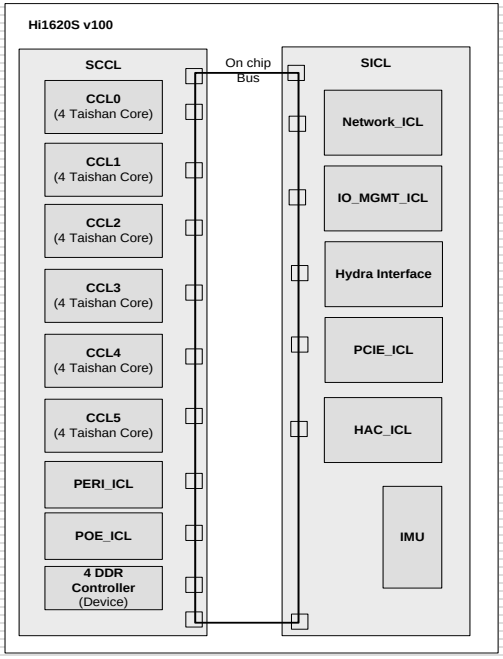


芯片架构-乐高架构

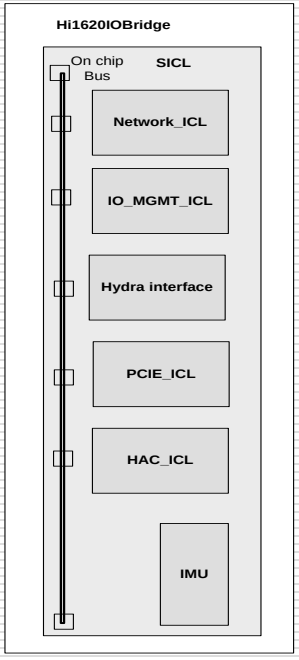
Hi1620:2 CPU DIE+1 I/O DIE



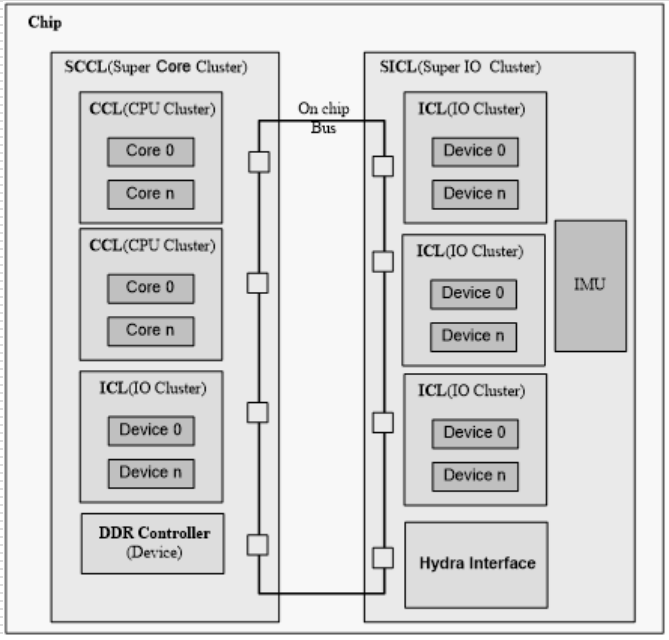
Hi1620s:1 CPU DIE+1 I/O DIE

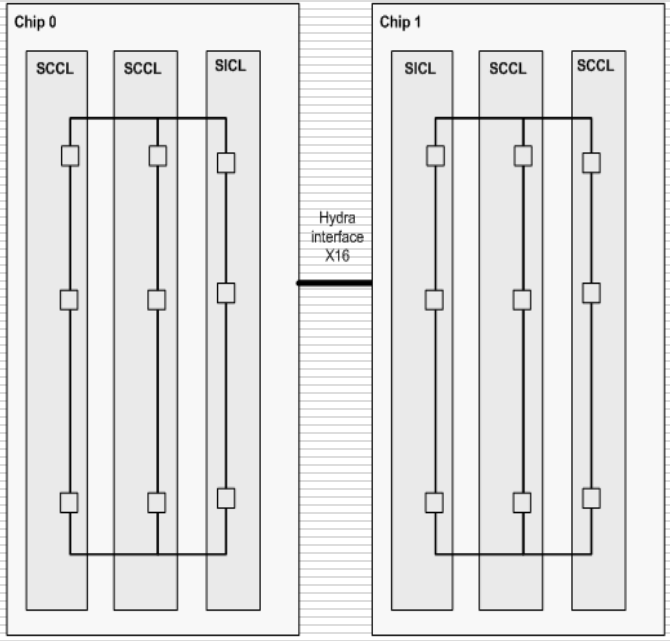


Hi1601:1 I/O DIE



鲲鹏920系列芯片架构

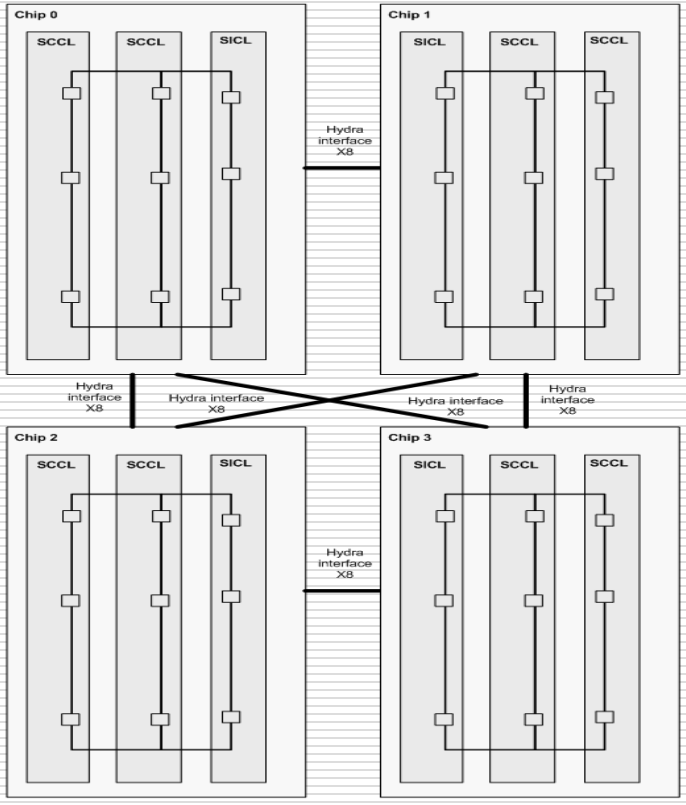




- 鲲鹏920处理器片上系统组成
- 2P服务器处理器多片互联方案

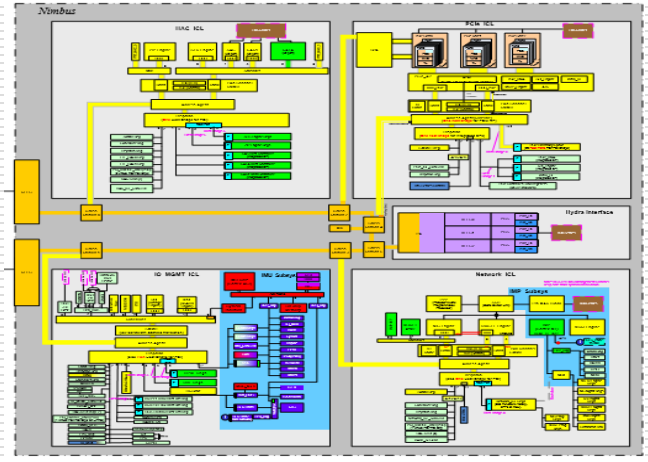
鲲鹏920系列芯片架构

- 4P服务器处理器多片互联方案示例如右图所示：每颗芯片各提供2个SCCL和1个SICL。芯片之间通过片间缓存一致接口连接，片间带宽高达240Gbps。

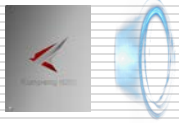


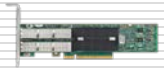
芯片架构– IO子系统

- 丰富的IO，PCIE化设计。
- 各子系统 PCIE（含CCIX），Hydra（多片互联），Network，Storage，HAC，ME。各自遵循行业标准，兼容软件接口规范，满足开源及演进要求。



SBSA	GIC: GICv3/v4；SMMU：SMMUv3.1 UART/Watchdog:
PCI-E	36 lanes of PCIe G4.0, 20 Root ports at max, Peer2Peer, ATS/PRI, CCIX
Networking（NIC、ROCE）	8 lanes of ETH, Combo MACs support 2x 100GE, 2x 50GE, 4x 25GE, 8x 10GE, 8x GE RoCEv2/RoCEv1 with programmable DC-QCN， long/normal Atomic, SR-IOV
Storage IO	x4 USB 3.0 EHCI/UHCI x16 SAS 3.0(STP supported) x2 SATA 3.0 AHCI
Crypto Engine	AES, DES/3DES, MD5,SHA1,SHA2,HMAC, CMAC Up to 50Gbps
Compression/ Decompression	GZIP, LZS, LZ4 Up to 40Gbps(compress)/100Gbps(decompress) Statefull and stateless
RAID	XOR/PQ/EC/in-line DIF acceleration
ME（IMU）	Isolated management subsystem. Co-works with BMC and provides firmware configuration of the server chip
Scale up	Coherent SMP interface for 2P/4P Up to 240Gbps per port




RoCE 网卡

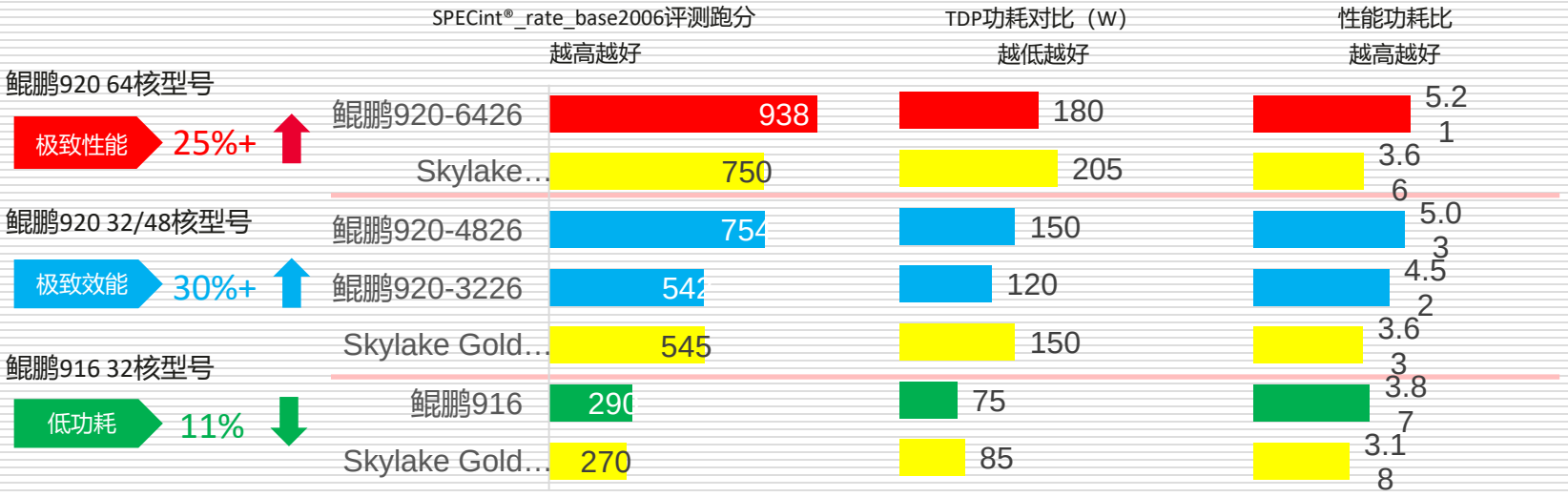
SAS 控制器

南桥


CPU

QAT

鲲鹏系列处理器 vs 业界主流产品的对比

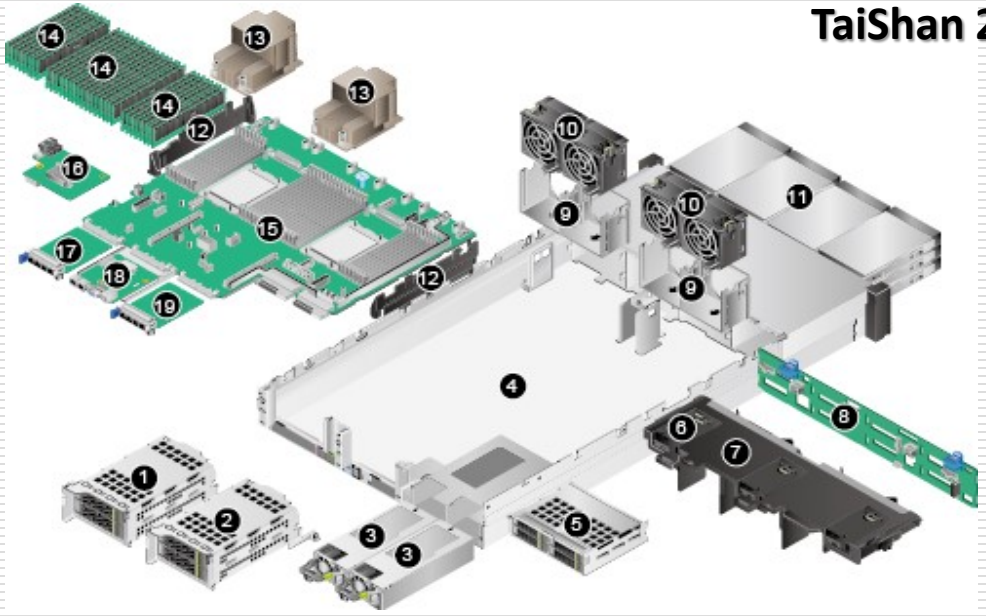


第5章 ARMv8处理器体系结构

5.3 鲲鹏处理器920

5.4 TaiShan200服务器

TaiShan服务器物理结构

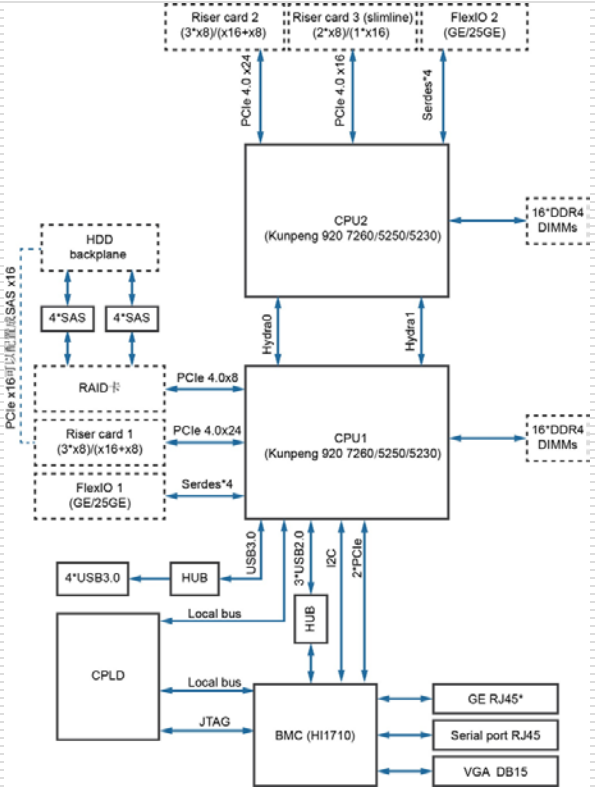


TaiShan 200服务器（型号2280）

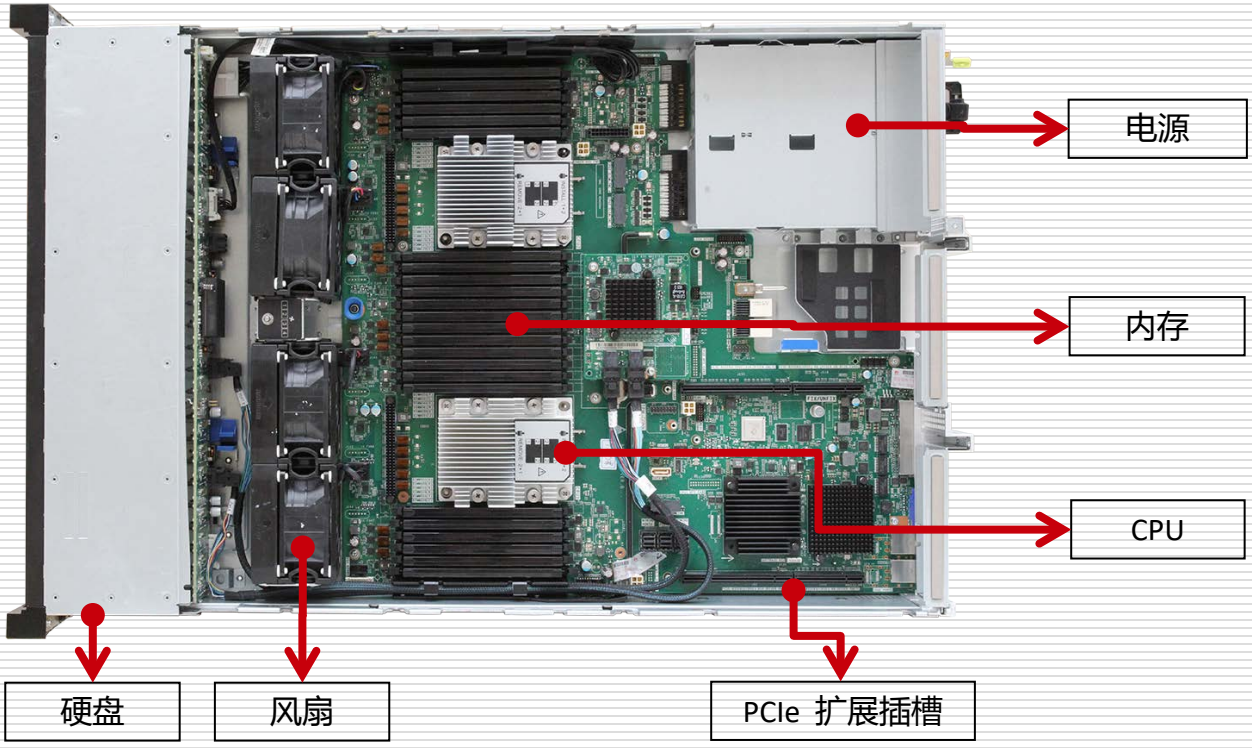
1	IO模组1	2	IO模组2
3	电源模块	4	机箱
5	IO模组3	6	超级电容支架
7	导风罩	8	前置硬盘背板
9	风扇支架	10	风扇模块
11	前置硬盘	12	理线架
13	散热器	14	DIMM
15	主板	16	RAID扣卡
17	灵活IO卡1（CPU 1）	18	iBMC插卡
19	灵活IO卡2（CPU 2）	-	-

TaiShan服务器逻辑结构

- 支持两路自研鲲鹏920 7260、5250、5240或5230处理器，每个处理器支持16个DDR4 DIMM。
- 以太网灵活插卡可支持2种插卡包括4*GE和4*25GE，通过CPU本身自带高速Serdes接口完成。
- RAID扣卡通过PCIe总线跟CPU1连接，RAID卡出SAS信号线缆跟硬盘背板连接，通过不同的硬盘背板可支持多种本地存储规格。
- BMC使用自研管理芯片Hi1710，可外出VGA、管理网口、调试串口等管理接口。



服务器内部视图



鲲鹏计算的典型应用场景

行业云



- 电信、政务、金审、金税等行业云
- 虚拟化性能优越
- 实时音视频极致体验

AI计算



- 鲲鹏+昇腾协同加速使能AI应用：人脸识别、车牌识别等

边缘计算



- AI边缘，5G移动边缘场景
- 电信、交通边缘计算场景

HPC



- 业界首款兼容ARM的四路服务器，超强算力
- 32个DDR4超大内存

Web应用



- 鲲鹏RSA安全加解密加速引擎
- 互联网Web HTTPS连接加速

运营商

基于鲲鹏电信云的VoLTE首呼

省际高速公路

车辆通行告别收费站

天文台

打造SKA区域中心原型系统

电商

互联网更快速的安全连接体验

第6章 ARMv8指系统



6.1 ARMv8寄存器

6.2 ARMv8寻址方式

6.3 ARMv8指令系统

基于ARMv8架构的处理器体系结构

- ARMv8寄存器：

位宽	分类		
32-bit	Wn（通用寄存器）	WZR（0寄存器）	WSP（堆栈指针寄存器）
64-bit	Xn（通用寄存器）	XZR（0寄存器）	SP（堆栈指针寄存器）

基于ARMv8架构的处理器体系结构

- AArch32重要寄存器：

寄存器类型	Bit	描述
R0-R14	32bit	通用寄存器，但是ARM不建议使用有特殊功能的R13，R14，R15当做通用寄存器使用。
SP_x	32bit	通常称R13为堆栈指针，除了User和Sys模式外，其他各种模式下都有对应的SP_x寄存器：x ={ und/svc/abt/irq/fiq/hyp/mon}
LR_x	32bit	称R14为链接寄存器，除了User和Sys模式外，其他各种模式下都有对应的SP_x寄存器：x ={ und/svc/abt/svc/irq/fiq/mon},用于保存程序返回链接信息地址，AArch32环境下，也用于保存异常返回地址，也就说LR和ELR是公用一个，AArch64下是独立的。
ELR_hyp	32bit	Hyp mode下特有的异常链接寄存器，保存异常进入Hyp mode时的异常地址。
PC	32bit	通常称R15为程序计算器PC指针，AArch32 中PC指向取指地址，是执行指令地址+8，AArch64中PC读取时指向当前指令地址。
CPSR	32bit	记录当前PE的运行状态数据,CPSR.M[4:0]记录运行模式，AArch64下使用PSTATE代替。
APSR	32bit	应用程序状态寄存器，EL0下可以使用APSR访问部分PSTATE值。
SPSR_x	32bit	是CPSR的备份，除了User和Sys模式外，其他各种模式下都有对应的SPSR_x寄存器：x ={ und/svc/abt/irq/fiq/hpy/mon}，注意：这些模式只适用于32bit运行环境。
HCR	32bit	EL2特有，HCR.{TEG,AMO,IMO,FMO,RW}控制EL0/EL1的异常路由。
SCR	32bit	EL3特有，SCR.{EA,IRQ,FIQ,RW}控制EL0/EL1/EL2的异常路由，注意EL3始终不会路由。
VBAR	32bit	保存任意异常进入非Hyp mode & 非Monitor mode的跳转向量基地址。
HVBAR	32bit	保存任意异常进入Hyp mode的跳转向量基地址。
MVBAR	32bit	保存任意异常进入Monitor mode的跳转向量基地址。
ESR_ELx	32bit	保存异常进入ELx时的异常综合信息，包含异常类型EC等，可以通过EC值判断异常class。
PSTATE		不是一个寄存器，是保存当前PE状态的一组寄存器统称，其中可访问寄存器有：PSTATE.{NZCV,DAIF,CurrentEL,SPSel},属于ARMv8新增内容，主要用于64bit环境下。

基于ARMv8架构的处理器体系结构

- AArch64重要寄存器：

寄存器类型	Bit	描述
X0-X30	64bit	通用寄存器，如果有需要可以当做32bit使用：WO-W30
LR (X30)	64bit	通常称X30为程序链接寄存器，保存跳转返回信息地址
SP_ELx	64bit	若PSTATE.M[0] ==1，则每个ELx选择SP_ELx，否则选择同一个SP_EL0
ELR_ELx	64bit	异常链接寄存器，保存异常进入ELx的异常地址（x={0,1,2,3}）
PC	64bit	程序计数器，俗称PC指针，总是指向即将要执行的下一条指令
SPSR_ELx	32bit	寄存器，保存进入ELx的PSTATE状态信息
NZCV	32bit	允许访问的符号标志位
DIAF	32bit	中断使能位：D-Debug，I-IRQ，A-SError，F-FIQ，逻辑0允许
CurrentEL	32bit	记录当前处于哪个Exception level
SPSel	32bit	记录当前使用SP_EL0还是SP_ELx，x= {1,2,3}
HCR_EL2	32bit	HCR_EL2.{TEG,AMO,IMO,FMO,RW}控制EL0/EL1的异常路由 逻辑1允许
SCR_EL3	32bit	SCR_EL3.{EA,IRQ,FIQ,RW}控制EL0/EL1/EL2的异常路由 逻辑1允许
ESR_ELx	32bit	保存异常进入ELx时的异常综合信息，包含异常类型EC等.
VBAR_ELx	64bit	保存任意异常进入ELx的跳转向量基地址 x={0,1,2,3}
PSTATE		不是一个寄存器，是保存当前PE状态的一组寄存器统称，其中可访问寄存器有：PSTATE.{NZCV,DAIF,CurrentEL,SPSel},属于ARMv8新增内容,64bit下代替CPSR

寄存器

ARM Low 32-bits	ARM Register	X86 Register	X86 Low 32-bits	X86 Low 16-bits	X86 Low 8-bits
W0	X0	%rax	%eax	%ax	%al
W0	X0	%rdi	%edi	%di	%dl
W1	X1	%rsi	%esi	%si	%sl
W2	X2	%rdx	%edx	%dx	%dl
W3	X3	%rcx	%ecx	%cx	%cl
W4	X4	%r8	%r8d	%r8w	%r8b
W5	X5	%r9	%r9d	%r9w	%r9b
W6	X6				
W7	X7				
W16	X16	%r10	%r10d	%r10w	%r10b
W17	X17	%r11	%r11d	%r11w	%r11b
WSP	SP	%rsp	%esp	%sp	%spl
W19	X19	%rbx	%ebx	%bx	%bl
W20	X20	%rbp	%ebp	%bp	%bpl
W21	X21	%r12	%r12d	%r12w	%r12b
W22	X22	%r13	%r13d	%r13w	%r13b
W23	X23	%r14	%r14d	%r14w	%r14b
W24	X24	%r15	%r15d	%r15w	%r15b

寄存器

Low 32-bits	ARM Register	Conventional use	X86 Register
W25	X25	Local variable, caller-owned(用于保存跨函数调用长生命周期数据，子函数避免使用)	
W26	X26	Local variable, caller-owned(用于保存跨函数调用长生命周期数据，子函数避免使用)	
W27	X27	Local variable, caller-owned(用于保存跨函数调用长生命周期数据，子函数避免使用)	
W28	X28	Local variable, caller-owned(用于保存跨函数调用长生命周期数据，子函数避免使用)	
	PC(不可直接访问)	Instruction pointer	%rip（可用于地址访问）
	%CPSR	Status/condition code bits	%eflags
W9…W18	X9…X18	Temporary registers (临时寄存器，子函数内可使用)	
	LR(r30)	The Link Register.	
	FP(r29)	The Frame Pointer	
WZR	XZR	Zero-register(read only)	
		Code Segment	CS
		Default Data Segment	DS
		Stack Segment	SS
		Extra Data Segment	ES
		Additional Data Segment	FS
		Additional Data Segment	GS

状态寄存器

AArch64	Flag Name	含义	X86
CF	Carry Flag	进位/借位Carry。如果算术指令的结果的最高有效位产生了进位/借位，则置1；否则置0。这个标志位指示了无符号整数计算的结果是否溢出	CF
	Parity Flag	奇偶校验位Parity。如果指令结果的最低字节含有偶数个比特1，则置1；否则置0。	PF
	Auxiliary Flag	辅助进位/借位Auxiliary Carry。如果算术运算在第3比特位（最低比特位是0）产生了进位/借位，则置1；否则置0。这个标志主要用于二进制编码的十进制数BCD算术运算。	AF
ZF	Zero Flag	零标志位Zero。如果指令结果为0，则置1；否则置0	ZF
NF(negative)	Sign Flag	符号位Sign。设置为指令结果的最高有效比特位，即有符号整数的符号比特位。0表示结果是正数或者是0；1表示负数。	SF
VF	Overflow Flag	溢出位Overflow。如果整数运算结果超出了目标操作数可容纳的值域（正数过大，负数过小，不包括符号位），则置1；否则置0（即结果可信）。这个标志指示了有符号整数算术运算的结果是否溢出。	OF
	DF Direction Flag		

ARM寻址方式（1）

- ARM处理器共有八种寻址方式：
  - 立即数寻址
  - 寄存器寻址
  - 寄存器间接寻址
  - 基址寻址
  - 多寄存器寻址
  - 堆栈寻址
  - 相对寻址
  - 寄存器移位寻址

ARM寻址方式（2）

- **立即数寻址：**
  - 立即数寻址指令中的地址码就是操作数本身，可以立即使用的操作数。其中，#0xFF000和#64都是立即数。如操作数是常量，用#表示常量；0x或&表示16进制数，否则表示十进制数。

例如：

@指令省略了第1个操作数寄存器。将立即数0xFF000(第2操作数)装入R0寄存器

```
MOV  R0,#0xFF000
```

@R0减64，结果放入R0

```
SUB  R0,R0,#64
```

ARM寻址方式（3）

- **寄存器寻址**
  - 操作数的值在寄存器中，指令执行时直接取出寄存器值来操作，寄存器寻址是根据寄存器编码获取寄存器内存储的操作数

例如：

@将R2的值存入R1　　在第1个操作数寄存器的位置存放R2编码

```
MOV R1,R2
```

@将R1的值减去R2的值，结果保存到R0在第2操作数位置，存放的是寄存器R2的编码

```
SUB R0,R1,R2
```

ARM寻址方式（4）

- **寄存器间接寻址**
  - 操作数从寄存器所指向的内存中取出，寄存地存储的是内存地址

例如：

@将R2指向的存储单元的数据读出，保存在R1中 R2相当于指针变量

```
LDR R1,[R2]
```

@将R1的值写入到R2所指向的内存

```
STR R1,[R2]
```

@将寄存器R1的值和R2指定的存储单元的内容交换

```
SWP R1,R1,[R2]
```

[R2]表示寄存器所指向的内存
LDR 指令用于读取内存数据
STR 指令用于写入内存数据

ARM寻址方式（5）

- **基址变址寻址**
  - 基址寄存器的内容与指令中的偏移量相加，得到有效操作数的地址，然后访问该地址空间，基址变址寻址分为三种：
    - 前索引，例如：

```
LDR  R0, [R1,#4]
```

@R1存的地址+4，访问新地址里面的值，放到R0；

- 自动索引，例如：

```
LDR  R0, [R1,#4]!
```

@在前索引的基础上，新地址回写进R1；（注：!表示回写地址）

- 后索引，例如：

```
LDR  R0 [R1],#4
```

@R1存的地址的内容写进R0，R1存的地址+4再写进R1；

ARM寻址方式（6）

- **多寄存器寻址**
  - 一条指令完成多个寄存器的传送，最多16个寄存器，也称为块拷贝寻址

例如：

@将R1指向的存储单元中的数据读写到R2～R7、R12中，然后R1自加1

```
LDMIA R1!,{R2-R7,R12}
```

@将寄存器R2～R7、R12的值保存到R1指向的存储单元中，然后R1自加1

注：基址寄存器不允许为R15，寄存器列表可以为R0～R15 的任意组合。这里R1没有写成[R1]!，是因为这个位不是操作数位，而是寄存器位

```
STMIA R1!,{R2-R7,R12}
```

LDMIA 和 STMIA 是块拷贝指令，LDMIA是从R1所指向的内存中读数据，STMIA是向R1所指向的内存写入数据

- 执行这类指令要考虑如下几个问题：
  - 基址寄存器指向原始地址有没有放一个有效值
  - 寄存器列表哪个寄存器被最先传送
  - 存储器地址增长方向
  - 指令执行完成后，基址寄存器有没有指向一个有效值

ARM寻址方式（7）

- **寄存器堆栈寻址**
  - 是按特定顺序存取存储区，按后进先出原则，使用专门的寄存器SP（堆栈指针)指向一块存储区

例如：

@将栈内的数据，读写到R2～R7、R12中，然后下一个地址成为栈顶

```
LDMIA SP!,{R2-R7,R12}
```

@将寄存器R2～R7、R12的值保存到SP指向的栈中，SP指向的是栈顶

```
STMIA SP!,{R2-R7,R12}
```

ARM寻址方式（8）

- **相对寻址**
  - 即读取指令本身在内存中的地址。是相对于PC内指令地址偏移后的地址。由程序计数器PC提供基准地址，指令中的地址码字段作为偏移量，两者相加后得到的地址即为操作数的有效地址

例如：

```
BL   ROUTE1                    @调用到ROUTE1 子程序
BEQ LOOP                       @条件跳转到LOOP 标号处
…
LOOP MOV R2,#2
…
ROUTE1
…
```

ARM指令集 (1)

GNU ARM汇编语言语法格式：

```
[<label>:][<instruction or directive or pseudo-instruction>} @comment
```

- instruction伪指令
- directive伪操作
- pseudo-instruction伪指令
- <label>: 为标号, GNU ARM汇编中，任何以冒号结尾的标识符都被认为是一个标号，而不一定非要在一行的开始
- comment为语句的注释

注意：

- ARM指令，伪指令，伪操作，寄存器名可以全部为大写字母，也可全部为小写字母，但不可大小写混用。
- 如果语句太长，可以将一条语句分几行来书写，在行末用“\”表示换行 (即下一行与本行为同一语句)，“\”后不能有任何字符，包含空格和制表符 (Tab)。

ARM指令集（2）

ARM指令集——指令格式

ARM指令的基本格式如下：

```
<opcode> {<cond>} {S}    <Rd> ,<Rn>{,<operand2>}
```

其中<>号内的项是必须的，{}号内的项是可选的。各项的说明如下：

opcode：指令助记符；　　**cond**：执行条件；

S：是否影响CPSR寄存器的值；

Rd：目标寄存器；　　**Rn**：第1个操作数的寄存器；

operand2：第2个操作数；

ARM指令集（3）

- GNU ARM汇编语言语法格式：
  - GNU ARM汇编特殊字符和语法
    - 代码行中的注释符号：‘@’
    - 整行注释符号：‘#’
    - 语句分离符号：‘;’
    - 立即数前缀：‘#’或 ‘$’

ARM指令集与X86指令集

- ARM指令集是固定大小，固定格式的指令编码。指令地址32bit 位宽，4字节对齐；
- X86体系结构是可变长度指令集体系结构。指令地址没有对齐要求。
  - 主要差异列举如下：
    - ARM是一种load-store架构：在RISC体系结构中很常见，这意味着数据处理指令不能直接对内存的内容进行操作，它们仅对寄存器进行操作。 反过来，加载和存储指令只能在寄存器和内存之间传输数据。 相比之下，IA-32指令集的数据处理指令可以直接在内存以及寄存器上处理数据。
    - IA-32支持访问I/O地址空间的单独I/O指令：X86的指令集包括一部分指令可以直接对IO地址空间进行操作。例如IN或者OUT，直接对I/O 端口进行数据读写。ARM没有等效功能，而是假定所有外围设备都在标准4GB地址空间内映射到内存里的。

ARM指令集与X86指令集

- 无法在ARM指令中嵌入任意32位地址
  - 因为ARM指令是固定32bit宽度的，所以不可能直接在arm指令中编码进32bit的地址。所有的内存访问，都是基于存放在一个寄存器中的地址进行索引的。换句话说，所有ARM的内存访问都是通过通用寄存器间接进行的。由于指令集的可变长度性质，在IA-32中可以在指令中嵌入32位地址。X86可以进行直接地址访问。
- ARM指令不能包含任意的32位常量（ARM不能直接生成32bit宽度的立即数）。
- 同上的原因。ARM操作长度较长的立即数时，需要通过mov/movk 多次进行生成。
- IA-32使用分段寻址模型
  - 所有IA-32内存/存储器访问都相对于段寄存器之一，因此必须首先设置这些访问。 较大的偏移量需要较大的指令才能对较大的常数进行编码。 ARM没有分段寻址的概念，并且没有等效的分段寄存器。
  - DS: Offset(base,index,scale) → offset(base)

ARM指令集与X86指令集

- ARM指令通常都有3个操作数；x86 指令存在隐含操作数（cpuid mul）
  - 大多数ARM数据处理指令采用三个操作数，所有这些操作数可以是寄存器，其中之一可以是常量。 相反，IA-32指令通常具有只有两个操作数，从而使它们本质上具有破坏性（即，其中一个操作数被结果覆盖）。 这使ARM在指令级别上更加灵活。
  - 例如，可以将两个寄存器加在一起并将结果放在第三个寄存器中使用一条指令。相同的操作在IA-32上需要两条指令。

IA-32	ARM
`mov cx, ax ; ca = ax` `add cx, bx ; cx = cx + bx`	`add r0, r1, r2 ; r0 = r1 + r2`

ARM指令集与X86指令集

- 许多复杂的IA-32指令都没有与ARM直接对应的指令
  - 因为x86的芯片电路相对设计复杂，可以支持较多复杂操作的指令。例如BCD转换和操作，点积，三角函数和对数函数等。
- ARM子例程通常不使用stack
  - ARM BL/B/B.cond 指令（用于子例程调用），是将返回地址防止在链接寄存器LR，而不是把地址放到栈里。同样，子例程返回之后的返回地址，是从LR里返回，而不是存储在栈上的。
  - X86 call 和Ret 指令，将返回地址放在堆栈上并分别取出。此外，提供了ENTER和LEAVE指令，以在进入和离开过程时创建和释放固定格式的堆栈帧。ARM没有与此等效的方法，因为从堆栈指针中添加固定值或从中减去固定值非常简单。

ARM指令集（4）

- 指令分类：

指令类型	说明
跳转指令	条件跳转、无条件跳转 (#imm、 register)指令
异常产生指令	系统调用类指令 (SVC、 HVC、 SMC)
系统寄存器指令	读写系统寄存器，如： MRS、 MSR指令 可操作PSTATE的位段寄存器
数据处理指令	包括各种算数运算、逻辑运算、位操作、移位 (shift) 指令
load/store内存访问指令	load/store {批量寄存器、单个寄存器、一对寄存器、非-暂存、非特权、独占}以及load-Acquire、 store-Release指令 (A64没有LDM/STM指令)
协处理指令	A64没有协处理器指令

ARM指令集（5）

- A64指令特点：

A64指令编码宽度固定32bit
31个（X0-X30）个64bit通用用途寄存器（用作32bit时是W0-W30），寄存器名使用5bit编码
PC指针不能作为数据处理指或load指令的目的寄存器，X30通常用作LR
移除了批量加载寄存器指令 LDM/STM, PUSH/POP, 使用STP/LDP 一对加载寄存器指令代替
增加支持未对齐的load/store指令立即数偏移寻址，提供非-暂存LDNP/STNP指令，不需要hold数据到cache中
没有提供访问CPSR的单一寄存器，但是提供访问PSTATE的状态域寄存器
相比A32少了很多条件执行指令，只有条件跳转和少数数据处理这类指令才有条件执行.
支持48bit虚拟寻址空间
大部分A64指令都有32/64位两种形式
A64没有协处理器的概念

ARM指令集（6）

- 跳转指令：
  - 条件跳转

指令	说明
B.cond	cond为真跳转
CBNZ	CBNZ X1，label　//如果X1!= 0则跳转到label
CBZ	CBZ X1，label　//如果X1== 0则跳转到label
TBNZ	TBNZ X1，#3 label　//若X1[3]!=0,则跳转到label
TBZ	TBZ X1，#3 label　//若X1[3]==0,则跳转到label

ARM指令集（7）

- 跳转指令：
  - 绝对跳转

指令	说明
B	绝对跳转
BL	绝对跳转 #imm，返回地址保存到LR（X30）
BLR	绝对跳转reg，返回地址保存到LR（X30）
BR	跳转到reg内容地址，
RET	子程序返回指令，返回地址默认保存在LR（X30）

例如：

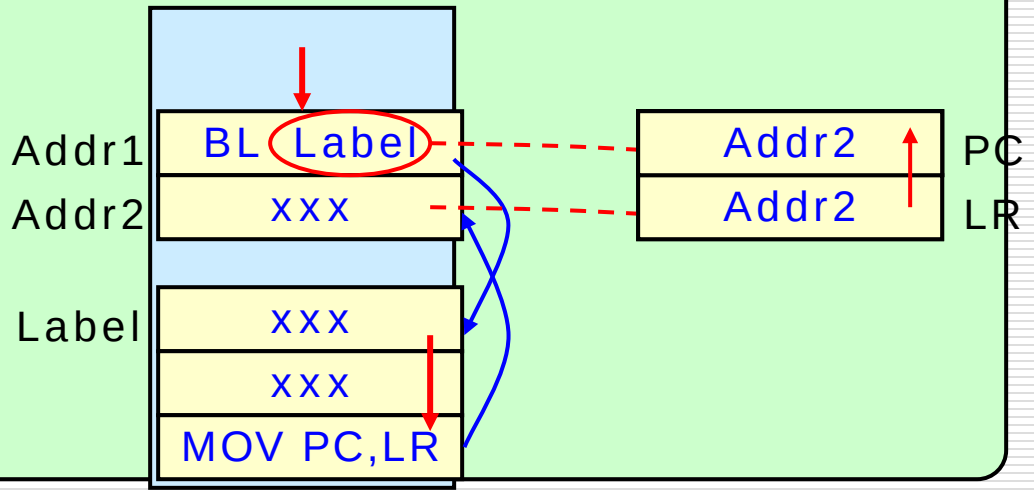
```
BL func                    @调用子程序func
    …
    func
    …
    MOV R15,R14    @子程序返回
```

ARM指令集（8）

带链接的分支指令——BL指令除了具有跳转功能，还能在跳转之前将下一条指令的地址拷贝到R14(即LR) 链接寄存器中，它适用于子程序调用。跳转范围限制在当前指令的±32M字节地址内。指令格式如下：

例如： BL DelayNS ;调用子程序DelayNS

2. 程序跳转到目标地址Label继续执行，当子程序执行结束后，将LR寄存器内容存入PC，返回调用函数继续执行



程序代码

ARM指令集（9）

- 异常产生和返回指令：

指令	说明
SVC	SVC系统调用，目标异常等级为EL1
HVC	HVC系统调用，目标异常等级为EL2
SMC	SMC系统调用，目标异常等级为EL3
ERET	异常返回，使用当前的SPSR_ELx和ELR_ELx

ARM指令集（10）

- 系统寄存器指令：

指令	说明
MRS	R <- S: 通用寄存器 <= 系统寄存器
MSR	S <- R: 系统寄存器 <= 通用寄存器

例如：

```
MRS R0,CPSR          @状态寄存器CPSR的值存入寄存器R0中

MSR CPSR_f,R0        @用R0的值修改CPSR的条件标志域

MSR CPSR_fsxc,#5     @CPSR的值修改为5
```

ARM指令集（11）

- 数据处理指令：

数据处理指令类型					
算数运算	逻辑运算	数据传输	地址生成	位段移动	移位运算
ADDS	ANDS	MOV	ADRP	BFM	ASR
SUBS	EOR	MOVZ	ADR	SBFM	LSL
CMP	ORR	MOVK		UBFM	LSR
SBC	MOVI			BFI	ROR
RSB	TST			BFXIL	
RSC				SBFIZ	
CMN				SBFX	
MADD				UBFIZ	
MSUB					
MUL					
SMADDL					
SDIV					
UDIV					

ARM指令集（12）

□ 算术运算指令：

指令	说明
ADDS	加法指令，若S存在，则更新条件位flag
ADCS	带进位的加法，若S存在，则更新条件位flag
SUBS	减法指令，若S存在，则更新条件位flag
SBC	将操作数 1　　减去操作数 2，再减去 标志位C的取反值，结果送到目的寄存器 Xt/Wt
RSB	逆向减法，操作数 2 –操作数 1，结果 Rd
RSC	带借位的逆向减法指令，将操作数 2 减去操作数 1，再减去 标志位C的取反值，结果送目标寄存器Xt/Wt
CMP	比较相等指令
CMN	比较不等指令
NEG	取负数运算，NEG X1，X2 // X1 = X2按位取反+1（负数=正数补码+1）
MADD	乘加运算
MSUB	乘减运算
MUL	乘法运算
SMADDL	有符号乘加运算
SDIV	有符号除法运算
UDIV	无符号除法运算

ARM指令集（13）

- 算术运算指令：

例如：

ADD 加法指令

```
ADD R0,R1, #5          @R0=R1+5
ADD R0,R1,R2           @R0=R1+R2
ADD R0,R1,R2,LSL #5    @R0=R1+R2左移5位
```

SUB 减法指令

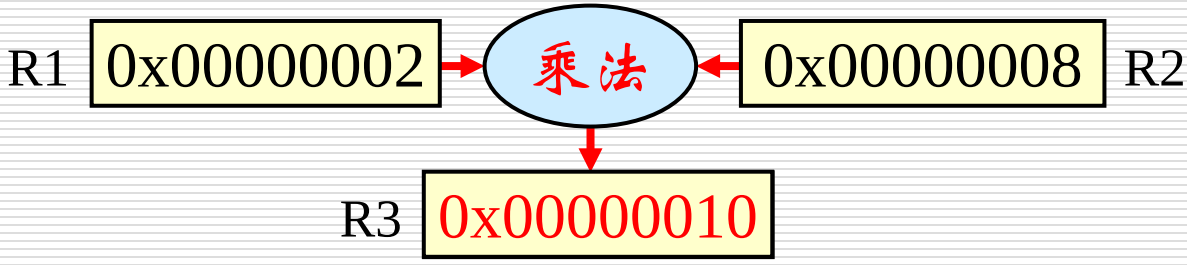
```
SUB R0,R1,R2           @R0=R1-R2
SUB R0,R1,R2,LSL #5    @R0=R1-R2左移5位
```

ARM指令集（14）

- ARM指令集——32×32位乘法指令

应用示例：

```
MUL  R3,R2,R1      ;R3=R2×R1
```



ARM指令集（15）

- 逻辑运算指令：

指令	说明
ANDS	按位与运算，如果S存在，则更新条件位标记
EOR	按位异或运算
ORR	按位或运算
TST	例如：TST W0，#0X40 //指令用来测试W0[3]是否为1,相当于：ANDS WZR,W0，#0X40

ARM指令集（16）

- 逻辑运算指令：

例如：

AND逻辑与指令

```
AND R0,R0, #5          @保持R0的第0位和第2位，其余位清0
```

ORR逻辑或指令

```
ORR R0,R0, #5          @R0的第0位和第2位设置为1，其余位不变
```

EOR逻辑异或指令

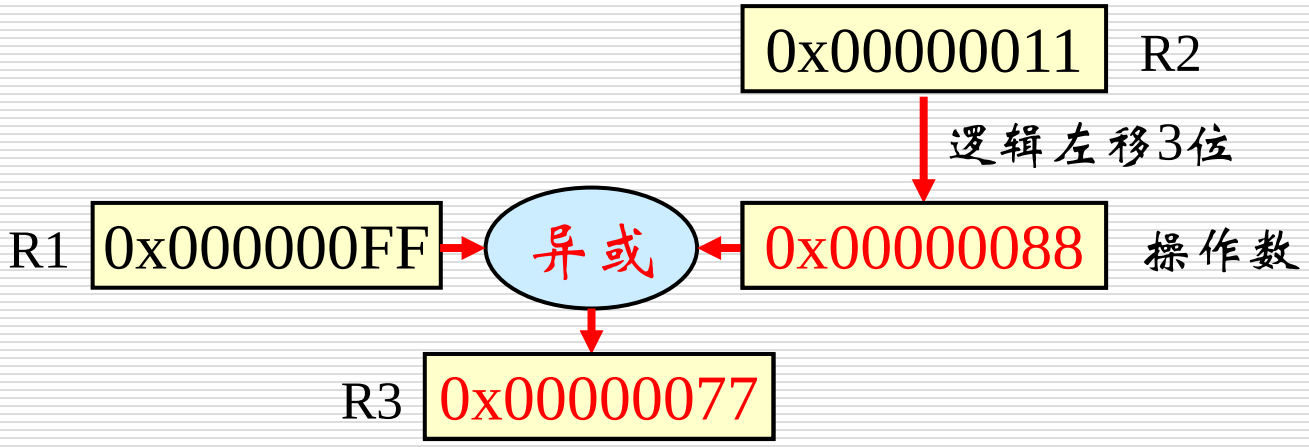
```
EOR R0,R0, #5          @R0的第0位和第2位取反，其余位不变
```

ARM指令集 (17)

- 算术逻辑运算指令

应用示例：

```
EOR  R3,R1, R2,LSL 0x03   ;R3=R1 ^ (R2 ×8)
```



ARM指令集（18）

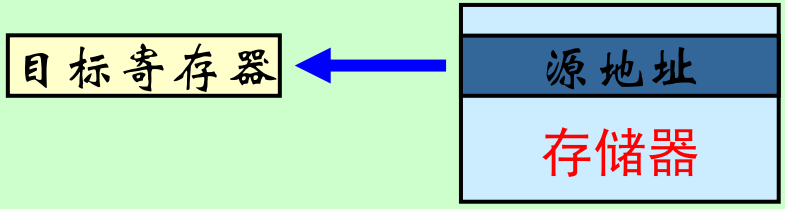
- 数据传输指令：

指令	说明
MOV	赋值运算指令
MOVZ	赋值#uimm16到目标寄存器Xd
MOVN	赋值#uimm16到目标寄存器Xd，再取反
MOVK	赋值#uimm16到目标寄存器Xd，保存其它bit不变
load/store	内存访问指令

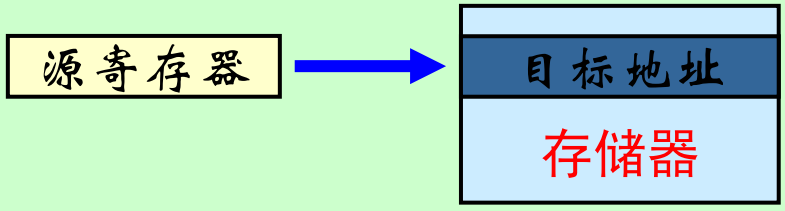
ARM指令集（19）

- ARM存储器访问指令——单寄存器存取

装载指令：**LDR** 目标寄存器,源地址



存储指令：**STR** 源寄存器,目标地址



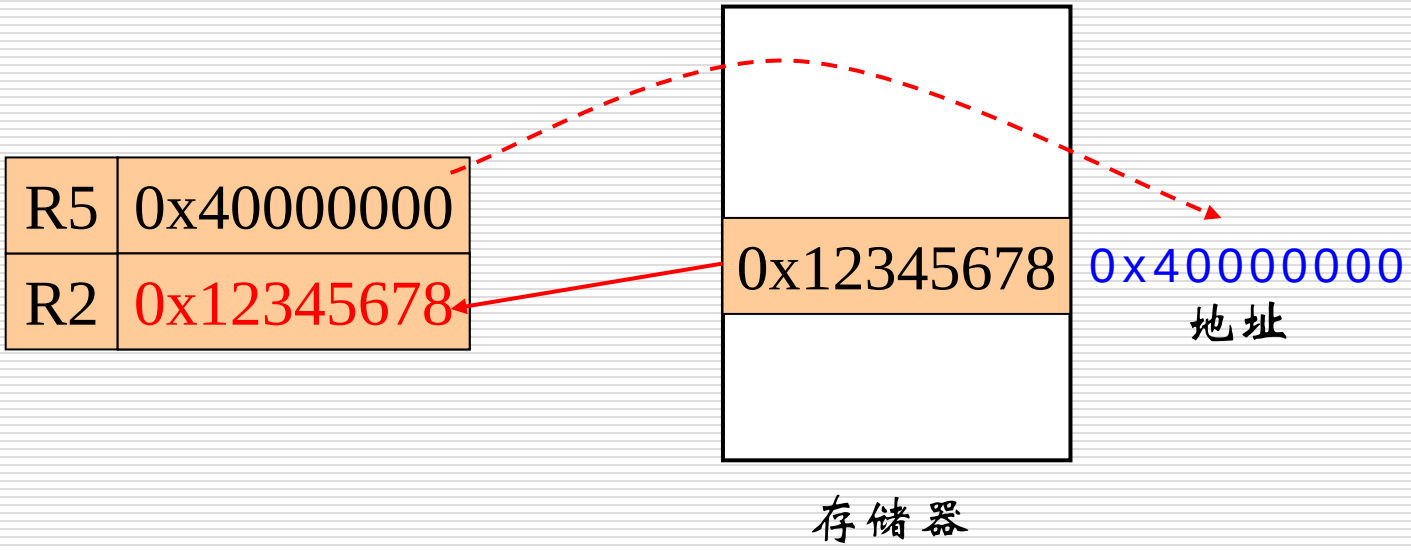
ARM指令集 (20)


装载指令：LDR 目标寄存器，源地址
保存指令：STR 源寄存器，目标地址
零偏移： 如：LDR Rd,[Rn]
前索引偏移： 如：LDR Rd,[Rn,#0x04]!
程序相对偏移： 如：LDR Rd,labe1
后索引偏移： 如：LDR Rd,[Rn],#0x04

ARM指令集 (21)

应用示例：

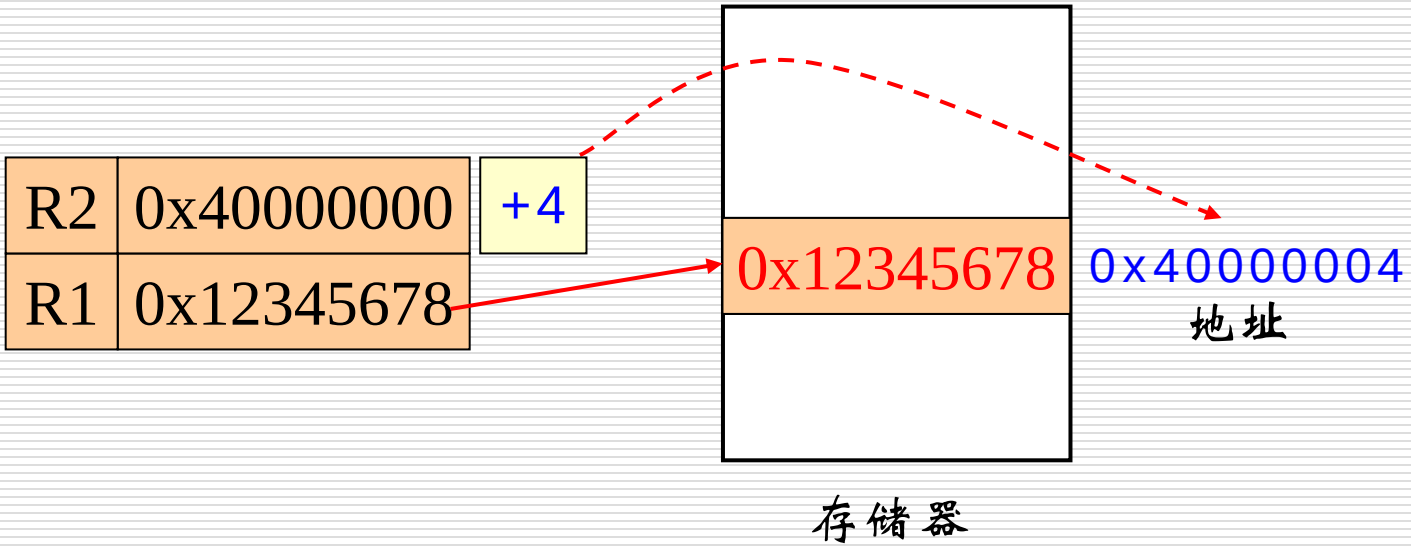
```
LDR    R2,[R5]          ;将R5指向地址的字数据存入R2
```



ARM指令集 (22)

应用示例：

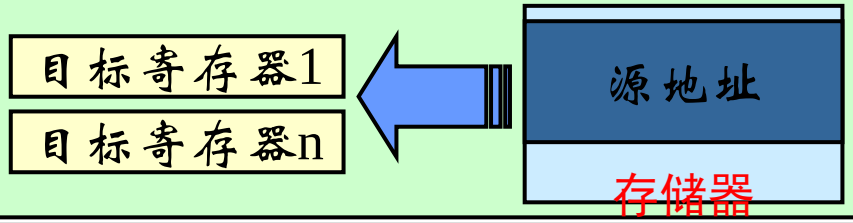
```
STR   R1,[R2,#0x04]    ;将R1的数据存储到R0+0x04地址
```



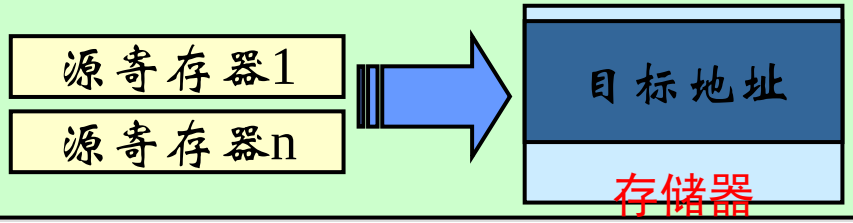
ARM指令集（23）

- ARM存储器访问指令——多寄存器存取

装载指令：**LDM**　　源地址,目标寄存器列表



存储指令：**STM**　　目标地址,源寄存器列表



ARM指令集 (24)

- ARM存储器访问指令——多寄存器存取

装载指令：LDMx

存储指令：STMx

LDM/STM指令搭配不同的后缀实现不同方式地址增长方式：

IA：每次传送后地址加4

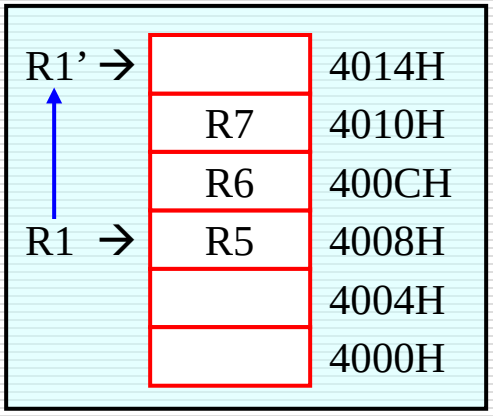
IB：每次传送前地址加4

DA：每次传送后地址减4

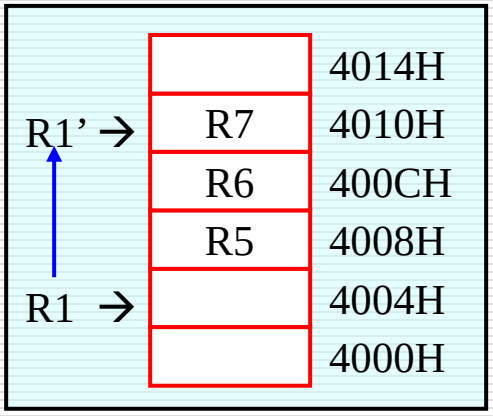
DB：每次传送前地址减4

ARM指令集 (25)

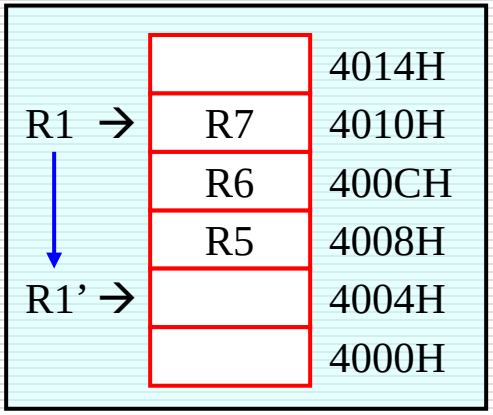
数据块传送指令操作过程如右图所示，其中R1为指令执行前的基址寄存器，R1’则为指令执行后的基址寄存器。



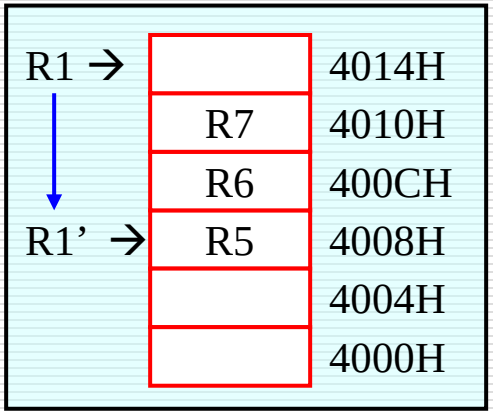
指令STMIA R1!,{R5-R7}



指令STMIB R1!,{R5-R7}



指令STMDA R1!,{R5-R7}



指令STMDB R1!,{R5-R7}

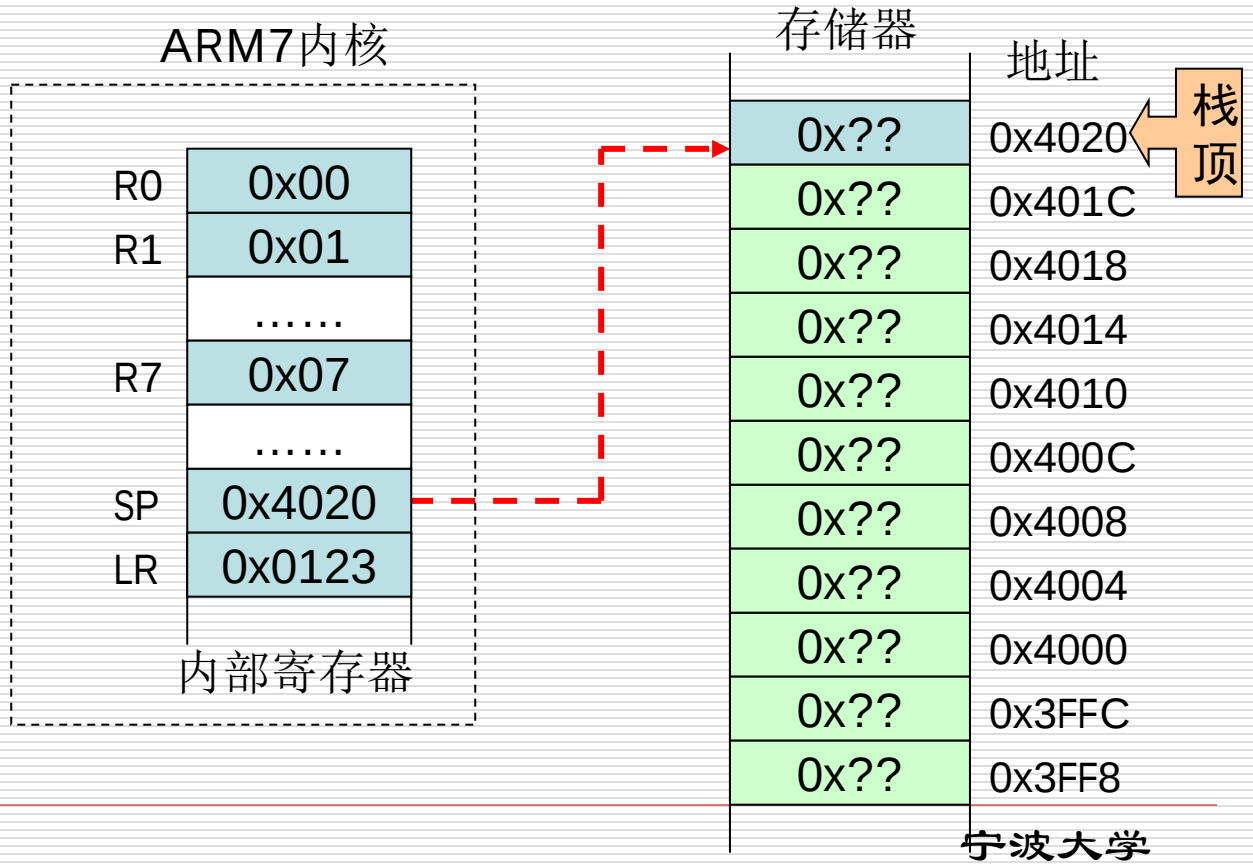
ARM指令集 (26)

- ARM存储器访问指令——满递减压栈操作

STMFD SP!,{R0-R7,LR}

1.压栈操作前寄存器和堆栈区的状态；

2.压栈操作前堆栈指针指向栈顶；



ARM指令集 (27)

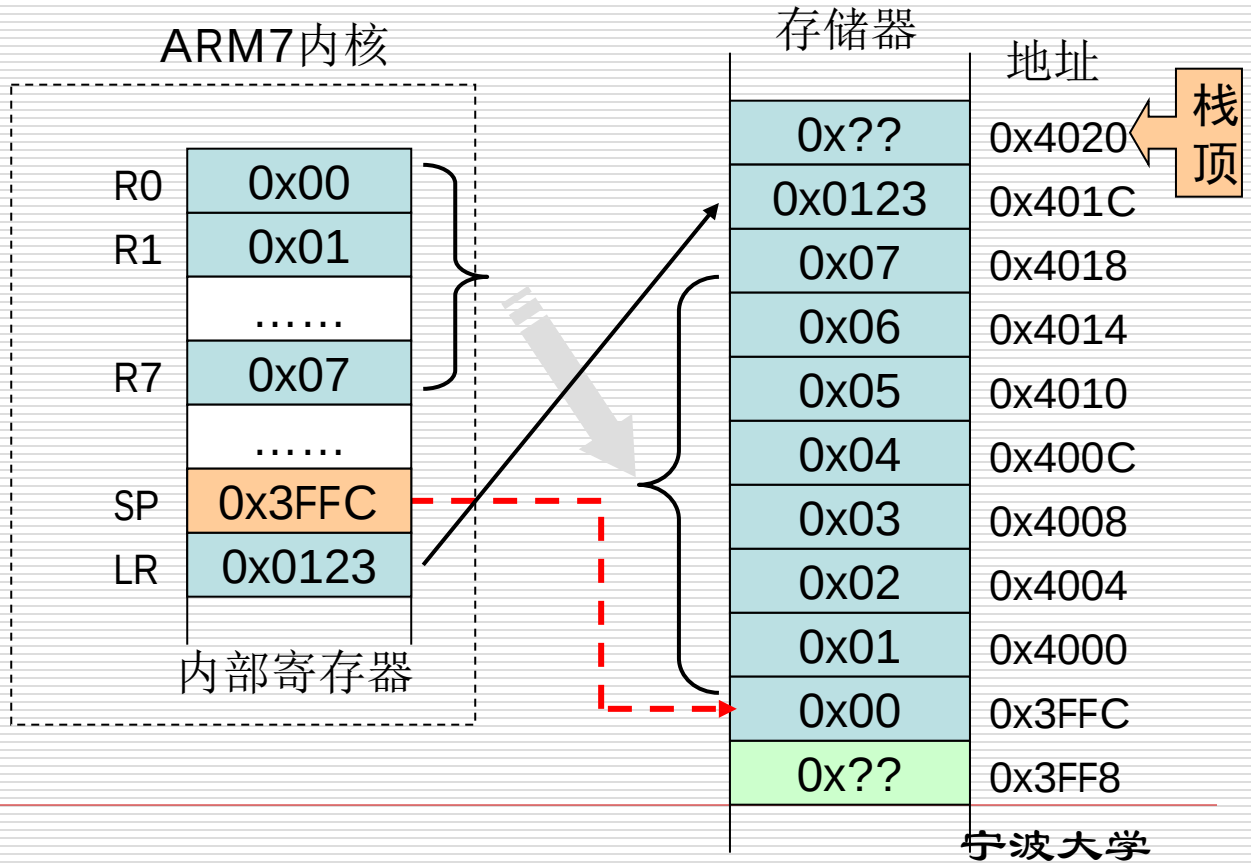
- ARM存储器访问指令——满递减压栈操作

STMFD SP!,{R0-R7,LR}

1.压栈操作前寄存器和堆栈区的状态；

2.压栈操作前堆栈指针指向栈顶；

3.执行压栈操作指令保存R0-R7和LR



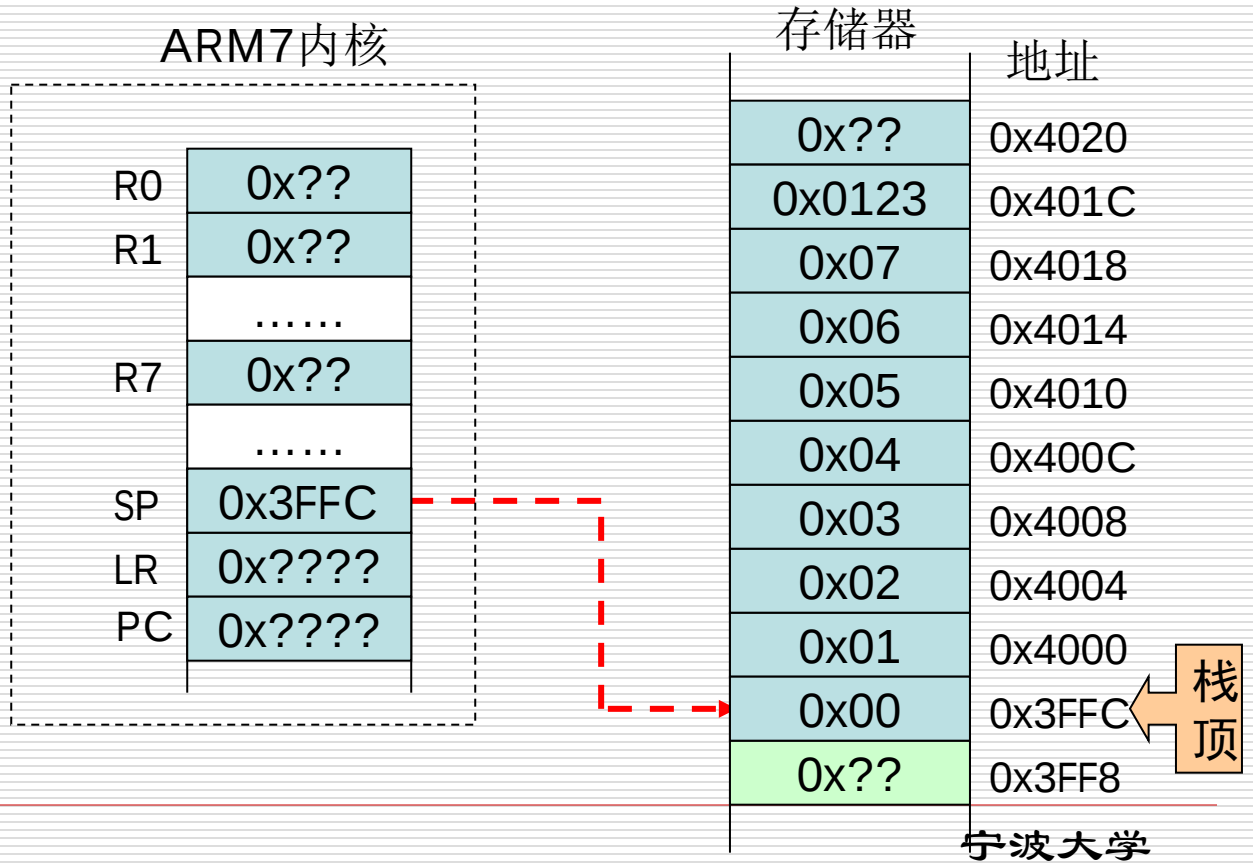
ARM指令集 (28)

- ARM存储器访问指令——满递减出栈操作

LDMFD SP!,{R0-R7,PC}

1.出栈操作前寄存器和堆栈区的状态；

2.出栈操作前堆栈指针指向栈顶；



ARM指令集 (29)

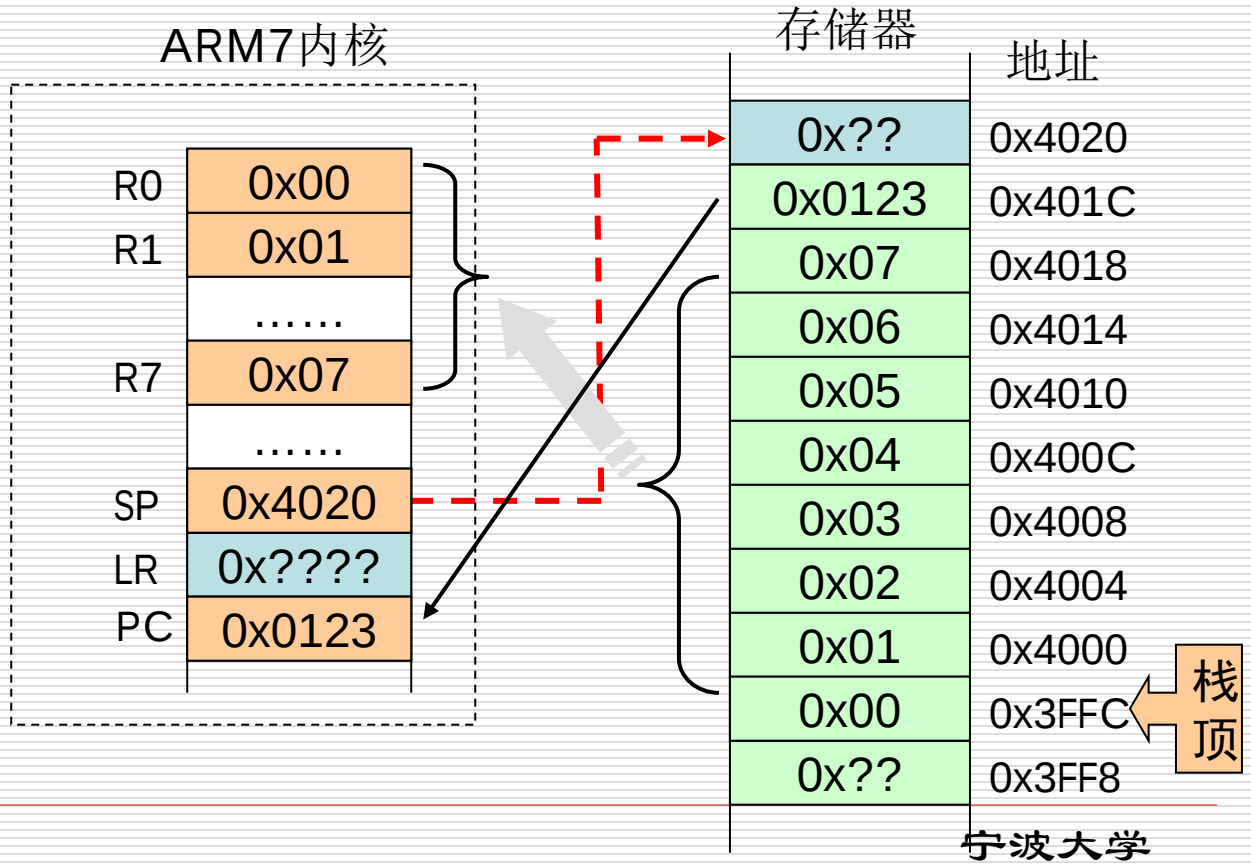
- ARM存储器访问指令——满递减出栈操作

LDMFD SP!,{R0-R7,PC}

1.出栈操作前寄存器和堆栈区的状态；

2.出栈操作前堆栈指针指向栈顶；

3.执行出栈操作指令恢复R0-R7和PC

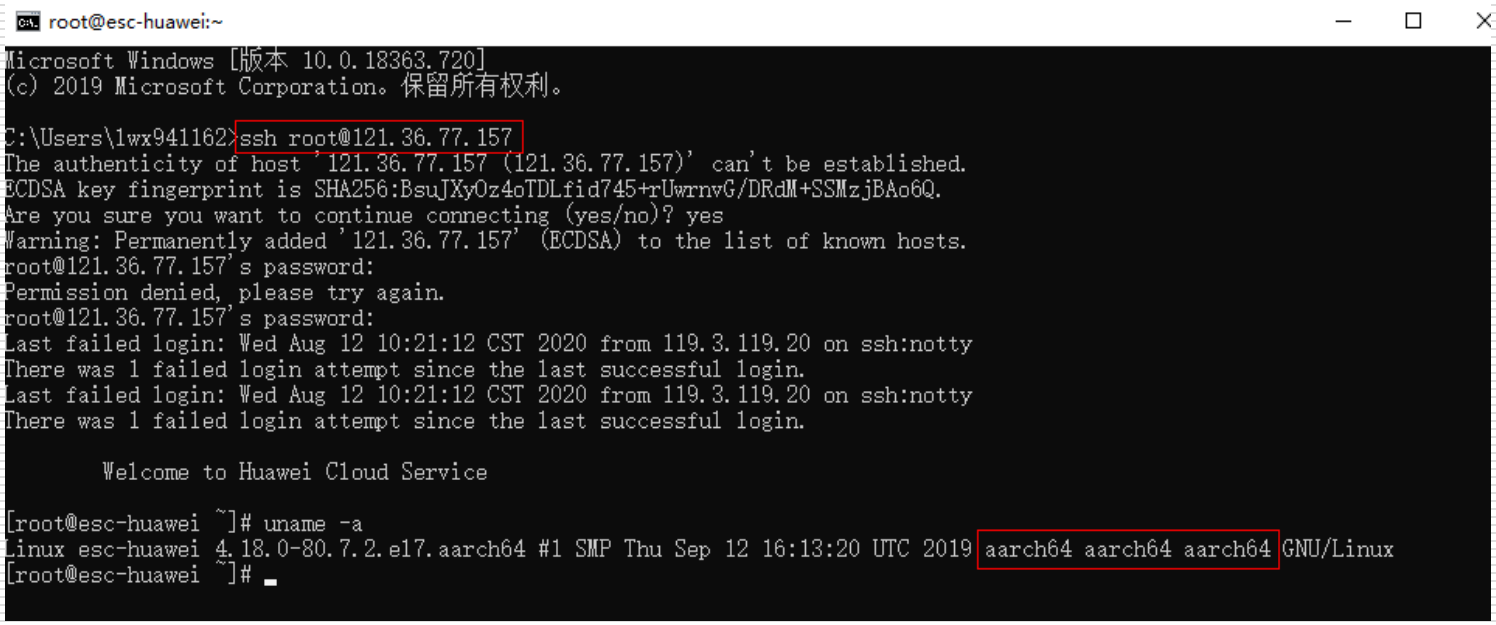


ARM编译与调试工具（1）

- 使用gcc编译器编译c文件流程
  - 预编译处理，生成main.i文件
  - 编译处理，生成main.s文件
  - 汇编处理，生成main.o文件
  - 链接处理，生成main.exe文件

ARM编译与调试工具（2）

- 使用SSH工具，输入用户名和密码，登陆



ARM编译与调试工具（3）

- 更新编译环境，命令如下：

```
yum groupinstall "Development tools“
```

- 升级gcc版本，依次如下命令：

```
yum -y install centos-release-scl
yum -y install devtoolset-7-gcc devtoolset-7-gcc-c++ devtoolset-7-
binutils
scl enable devtoolset-7 bash
echo "source /opt/rh/devtoolset-7/enable" >>/etc/profile
```

ARM编译与调试工具（4）

- 使用“gcc -v”命令可以输出gcc版本，过程截图如下：

```
[root@ecs-huawei2 ~]# scl enable devtoolset-7 bash
[root@ecs-huawei2 ~]# echo "source /opt/rh/devtoolset-7/enable" >>/etc/profile
[root@ecs-huawei2 ~]# gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/opt/rh/devtoolset-7/root/usr/libexec/gcc/aarch64-redhat-linux/7/lto-wrapper
Target: aarch64-redhat-linux
Configured with: ../configure --enable-bootstrap --enable-languages=c,c++,fortran,lto --prefix=/opt/rh/devtoolset-7/root/usr --mandir=/opt/rh/devtoolset-7/root/usr/
share/man --infodir=/opt/rh/devtoolset-7/root/usr/share/info --with-bugurl=http://bugzilla.redhat.com/bugzilla --enable-shared --enable-threads=posix --enable-check
ing=release --enable-multilib --with-system-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-gnu-unique-object --enable-linker-build-id --with-gcc
-major-version-only --enable-plugin --with-linker-hash-style=gnu --enable-initfini-array --with-default-libstdcxx-abi=gcc4-compatible --with-isl=/builddir/build/BUI
LD/gcc-7.3.1-20180303/obj-aarch64-redhat-linux/isl-install --disable-libmpx --enable-gnu-indirect-function --build=aarch64-redhat-linux
Thread model: posix
gcc version 7.3.1 20180303 (Red Hat 7.3.1-5) (GCC)
```

ARM编译与调试工具（5）

- 使用gcc来编译c语言程序
  - 新建汇编源文件，命令如下

```
vim hello.s
```

  - 在hello.s中输入以下代码：

```
.text
.global _start
_start:
        mov x0,#0
        ldr x1,=msg
        mov x2,len
        mov x8,64
        svc #0

        mov x0,123
        mov x8,93
        svc #0

.data
msg:
        .ascii "Hello World!\n"
len=.-msg
```

ARM编译与调试工具（6）

- 保存hello.s文件，然后通过运行以下命令将其编译为二进制文件

使用以下命令进行链接，输出可执行文件

```
as hello.s -o hello.o
```

使用以下命令执行hello程序

```
ld hello.o -o hello
```

```
./hello
```

ARM编译与调试工具（7）

- 程序运行结果如下图所示：

```
[root@ecs-huawei2 ~]# vim hello.s
[root@ecs-huawei2 ~]# as hello.s -o hello.o
[root@ecs-huawei2 ~]# ld hello.o -o hello
ld: warning: cannot find entry symbol _start; defaulting to 00000000004000b0
[root@ecs-huawei2 ~]# ls
hello  hello.o  hello.s
[root@ecs-huawei2 ~]# ./hello
Hello World!
```

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2020 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

